

COMPETITIVE PROGRAMMING LIBRARY

Eduardo Belian | Deixa de ser BOT

Compiled on: 2026-08-27

INDEX

00-GERAL

Conversoes.cpp	02
Limites.txt	02
Math.txt	02
Modelo.cpp	02

01-SDF

01-vector_demo.cpp	03
02-string_demo.cpp	03
03-queue_demo.cpp	03
04-map_demo.cpp	04
05-set_demo.cpp	05
06-adapters_demo.cpp	05
07-utilities_demo.cpp	06
08-algorithms_demo.cpp	06
09-loops_demo.cpp	06
10-advanced_techniques_demo.cpp	07
11-buscas_binarias_demo.cpp	08
12-MinElement_MaxElement.cpp	09
13-list_demo.cpp	09

02-BUSCAS

00-BuscaBinaria.cpp	10
01-Selecionar-Eventos.cpp	10
02-Selecionar-Eventos_K-pessoas.cpp	10
03-Stack_Monotonica.cpp	10
04-Psum2D.cpp	10
05-SparseTable.cpp	11
06-SparseTable2D.cpp	11

03-GRAFOS_1

01-MatrizAdj-SemPeso.cpp	13
02-MatrizAdj-ComPeso.cpp	13
03-ListaAdj-SemPeso.cpp	13
04-ListaAdj-ComPeso.cpp	13
05-PecorrerGrid.cpp	13
06-DFS.cpp	13
07-BFS.cpp	13
08-BFS-Grid.cpp	13
09-Bicoloridade.cpp	14
10-Toposort.cpp	14
11-BFS-Multisource.cpp	14
12-Find-Cycles.cpp	14
13-Diametro-Arvore.cpp	15
14-CompConexos-FloodFill.cpp	15

04-GRAFOS_2

01-Dijkstra.cpp	16
02-Dijkstra_Sem_PQ.cpp	16
03-Floyd-Warshall.cpp	16
04-Bellman-Ford.cpp	16
05-Modelagem.cpp	16
06-DSU.cpp	17
07-MST-Kruskal.cpp	17
Infos.txt	17

05-DP_1

01-LIS.cpp	18
02-LCS.cpp	18
03-SubsetSum.cpp	18
04-Knapsack.cpp	18
05-Reconstrucao.txt	19
06-ValorEsperado.cpp	19
06-ValorEsperado2.cpp	19

06-DP_2

01-Operacoes.txt	20
02-DP_Subconjuntos.cpp	20
03-Backtracking_TSP.cpp	20
04-TSP_Recursivo.cpp	20
05-TSP_Iterativa.cpp	21

07-TEORIADOSNUMEROS

00-Matrix.cpp	22
00-Matrix_Mod.cpp	22
01-Divisores.cpp	22
02-Fatoracao_Prima.cpp	22
03-Crivo.cpp	22
04-Crivos_Especiais.txt	22

05-Aritmetica_Modular.cpp	23
06-FastExp.cpp	23
07-Mat_FastExp.cpp	23
08-Combinatoria_Modular.cpp	23
09-NTT.cpp	24

08-RANGEQUERY

01-MergeSort.cpp	25
02-Merge_Inversoes.cpp	25
03-SegmentTree.cpp	25
04-FenwickTree.cpp	25

09-STRINGS

01-TRIE.cpp	27
01-TRIE_zSamuel.cpp	27
02.1-Hash.cpp	27
02.2-Hash_2x.cpp	27
03.1-Hash_String.cpp	28
03.2-Hash_String_2x.cpp	28
04-KMP.cpp	28
05-PiArray.cpp	29

10-GEOMETRIA

01-Pontos.cpp	30
02-Convex_Hull.cpp	30
03-ponto_no_poligono.cpp	31
04-triagulacao_poligono.cpp	31
05-area_entre_triangulos.cpp	32

MISC

BinarySearch.cpp	33
Divisibilidade.txt	33
GF.txt	33
JulianDay.cpp	33
LatLong.cpp	33
Random.cpp	33
TernarySearch.cpp	34
permutation-int.cpp	34

STRESS_TEST

brute.cpp	35
gen.cpp	35
solve.cpp	35
stressTest.sh	35

00 - GERAL

Conversoes.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
#define pii pair<int,int>
#define FORi(n) for(int i=0;i<n;i++)
#define FORj(n) for(int j=0;j<n;j++)
#define all(a) (a).begin(),(a).end()
#define printn(v,n) for(int i=0;i<n;i++) cout << v[i] << (i == n-1 ?
"\n" : " ")

ll fexp(ll base, ll pot) {
    ll ans = 1;
    while(pot) {
        if(pot & 1) { ans = ans * base; }
        base = base * base;
        pot >>= 1;
    }
    return ans;
}

ll bin_str_to_ll(string& s) {
    ll x = 0; int tamanho = s.size();
    for(int i=0;i<tamanho;i++) { if(s[i]=='1') { x += (ll(1) << (tam
anho-i-1)); } }
    return x;
}

string ll_to_str_bin(ll n, int sz=0) {
    if(n==0) { return "0"; }
    string res = "";
    int idx = 0;
    while(n > 0) {
        if(n & (ll(1) << idx)) { res += '1'; n -= (ll(1) << idx); }
        else { res += '0'; }
        idx++;
    }
    // Preenche com 0's
    FORi(sz-res.size()) { res += '0'; }
    reverse(all(res));
    return res;
}

ll hex_str_to_ll(string& s) {
    ll x = 0; int tamanho = s.size();
    for(int i=0;i<tamanho;i++) {
        if(s[i] <= '9') {
            x += fexp(16, (tamanho-i-1)) * (s[i]-'0');
        }
        else {
            x += fexp(16, (tamanho-i-1)) * (s[i]-'a'+10);
        }
    }
    return x;
}

string ll_to_str_hex(ll n) {
    if(n==0) { return "0"; }
    string res = "";
    while(n > 0) {
        int d = n%16;
        if(d < 10) { res += '0'+d; }
        else { res += 'a'+d-10; }
        n /= 16;
    }
    reverse(all(res));
    return res;
}

void solve() {
    string s, tipo; cin >> s >> tipo;

    if(tipo == "bin") {
        ll v = bin_str_to_ll(s);
        cout << v << " dec\n";
        cout << ll_to_str_hex(v) << " hex\n";
    }
    else if(tipo == "dec") {
        ll v = stoll(s);
        cout << ll_to_str_hex(v) << " hex\n";
        cout << ll_to_str_bin(v) << " bin\n";
    }
    else if(tipo == "hex") {
        ll v = hex_str_to_ll(s);
        cout << v << " dec\n";
        cout << ll_to_str_bin(v) << " bin\n";
    }
}

int main() {
    ios::sync_with_stdio(false); cin.tie(0);

    int x = 0;
    int t; cin >> t; while(x<t) {
        cout << "Case " << x+1 << ":\n";
        solve();
        cout << "\n";
        x++;
    }
}
```

```
int main() {
    ios::sync_with_stdio(false); cin.tie(0);

    int x = 0;
    int t; cin >> t; while(x<t) {
        cout << "Case " << x+1 << ":\n";
        solve();
        cout << "\n";
        x++;
    }
}
```

Limites.txt

-----+-----		
Tipo	O que armazena	Tamanho Típico
-----+-----		
int	Números inteiros	4 bytes
aprox. -2×10 ⁹ a 2×10 ⁹		
long long	Inteiros muito grandes	8 bytes
aprox. -9×10 ¹⁸ a 9×10 ¹⁸		
char	Um único caractere	1 byte
Caracteres ASCII		
bool	Verdadeiro ou falso	1 byte
true ou false		
float	Números com ponto flutuante	4 bytes
Precisão de ~7 dígitos		
double	Ponto flutuante com dupla precisão	8 bytes
Precisão de ~15 dígitos		
-----+-----		

Math.txt

```
// CEIL DE INTS
Ceil de a/b: (a + b - 1) / b

// TERMO GERAL DA P.A
An = A1 + (n-1).r

// SOMA DA P.A
Sn = ((A1 + An) . n) / 2

// TERMO GERAL DA P.G
An = A1 * q^(n-1)

// SOMA DA P.G
Sn = A1 * (q^n - 1) / (q - 1)

// P.G INFINITA (0 <= q < 1)
Sn = A1 / (1-q)

// MDC de a,b = MDC de a-b, b
gcd(a,b) = gcd(a-b,b)
```

Modelo.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
#define pii pair<int,int>
#define FORi(n) for(int i=0;i<n;i++)
#define FORj(n) for(int j=0;j<n;j++)
#define all(a) (a).begin(),(a).end()
#define printn(v,n) for(int i=0;i<n;i++) cout << v[i] << (i == n-1 ?
"\n" : " ")

const int MAXN = 1e5 + 7;
vector<int> adj[MAXN];

void solve() {
    int n,m; cin >> n >> m;

    vector<int> A(n);
    FORi(n) { cin >> A[i]; }
}

int main() {
    ios::sync_with_stdio(false); cin.tie(0);

    // cout << fixed << setprecision(9); *****

    int t; cin >> t; FORi(t) { solve(); }
}
```

01-SDF

01-vector_demo.cpp

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    cout << "--- Demonstração vector ---" << endl;

    // --- 1. Formas de Inicialização ---
    vector<int> v1; // Vazio
    vector<int> v2(5); // 5 elementos, inicializados com 0
    vector<int> v3(5, 10); // 5 elementos, todos com valor 10
    vector<int> v4 = {1, 2, 3, 4, 5}; // C++11 list initialization
    vector<int> v5(v4); // Cópia de outro vetor

    // --- 2. Funções Essenciais (Modificadores) ---
    cout << "\n--- Modificadores ---" << endl;
    v1.push_back(10); // Adiciona ao final [10]
    v1.push_back(20); // [10, 20]
    v1.push_back(30); // [10, 20, 30]

    cout << "v1 após push_back: ";
    for (int x : v1) { cout << x << " "; }
    cout << endl;

    v1.pop_back(); // Remove o último elemento [10, 20]
    cout << "v1 após pop_back: ";
    for (int x : v1) { cout << x << " "; }
    cout << endl;

    // insert(iterador_posicao, valor)
    v1.insert(v1.begin() + 1, 15); // Inserir 15 na posição 1: [10,
15, 20]
    cout << "v1 após insert: ";
    for (int x : v1) { cout << x << " "; }
    cout << endl;

    // erase(iterador_posicao)
    v1.erase(v1.begin() + 1); // Remove o elemento na posição 1:
[10, 20]
    cout << "v1 após erase: ";
    for (int x : v1) { cout << x << " "; }
    cout << endl;

    // --- 3. Acesso a Elementos ---
    cout << "\n--- Acesso ---" << endl;
    cout << "v4[0]: " << v4[0] << endl; // Acesso direto (rápido,
mas não verifica limites)
    cout << "v4.at(1): " << v4.at(1) << endl; // Acesso com
verificação (lança exceção se fora dos limites)
    cout << "Primeiro elemento (front): " << v4.front() << endl;
    cout << "Último elemento (back): " << v4.back() << endl;

    // --- 4. Tamanho e Capacidade ---
    cout << "\n--- Tamanho ---" << endl;
    cout << "Tamanho (size): " << v4.size() << endl;
    cout << "Está vazio? (empty): " << (v1.empty() ? "Sim" : "Não")
<< endl;

    v1.clear(); // Remove todos os elementos
    cout << "Tamanho de v1 após clear: " << v1.size() << endl;

    return 0;
}
```

02-string_demo.cpp

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    cout << "--- Demonstração string ---" << endl;

    // --- 1. Inicialização ---
    string s1; // Vazia
    string s2 = "Hello";
    string s3("World");
    string s4(5, '!'); // "!!!!!"
    string s5 = s2; // Cópia

    // --- 2. Funções Essenciais (Modificadores) ---
    cout << "\n--- Modificadores ---" << endl;

    // Concatenação
    string s_ola = s2 + " " + s3; // "Hello World"
    s_ola.append("!!!"); // "Hello World!!!"
    cout << "Concatenada: " << s_ola << endl;
```

```
// Tamanho
cout << "Tamanho (size): " << s_ola.size() << endl;
cout << "Tamanho (length): " << s_ola.length() << endl; //
Sinônimos

// Acesso
cout << "Primeiro char: " << s_ola[0] << endl; // 'H'
cout << "Último char: " << s_ola[s_ola.size() - 1] << endl; //
'!'

// Substring (substr): (posição_inicial, tamanho_da_substring)
string sub = s_ola.substr(6, 5); // "World"
cout << "Substring: " << sub << endl;

// Busca (find): retorna a posição ou string::npos se não
encontrar
size_t pos = s_ola.find("World");
if (pos != string::npos) {
    cout << "'World' encontrado na posição: " << pos << endl;
} else {
    cout << "'World' não encontrado." << endl;
}

// Modificação
s_ola.insert(6, "C++ "); // "Hello C++ World!!"
cout << "Após insert: " << s_ola << endl;

s_ola.erase(6, 4); // Remove 4 chars a partir da posição 6:
"Hello World!!"
cout << "Após erase: " << s_ola << endl;

s_ola.replace(6, 5, "Universo"); // "Hello Universo!!"
cout << "Após replace: " << s_ola << endl;

s_ola.clear();
cout << "Após clear, s_ola está vazia? " << (s_ola.empty() ? "Si
m" : "Não") << endl;

// --- 3. Conversões (String <-> Numérico) ---
cout << "\n--- Conversões ---" << endl;

// Numérico para String
int ano = 2025;
double pi = 3.14159;
string str_ano = to_string(ano);
string str_pi = to_string(pi);
cout << "Int para String: " << str_ano << endl;
cout << "Double para String: " << str_pi << endl;

// String para Numérico (C++11)
string s_num = "12345";
string s_float = "98.76";

try {
    int i_num = stoi(s_num); // String to Int
    double d_float = stod(s_float); // String to Double

    cout << "String para Int: " << i_num << endl;
    cout << "String para Double: " << d_float << endl;

    // Exemplo de erro
    // stoi("abc");
} catch (const invalid_argument& e) {
    cerr << "Erro de conversão (formato inválido): " << e.what()
<< endl;
} catch (const out_of_range& e) {
    cerr << "Erro de conversão (fora do range): " << e.what() <<
endl;
}

// Método clássico (via Stringstream)
stringstream ss;
ss << "54321 3.14"; // Coloca coisas na stream
int i_val;
double d_val;
ss >> i_val >> d_val; // Extraí coisas da stream
cout << "Stringstream (int): " << i_val << endl;
cout << "Stringstream (double): " << d_val << endl;

return 0;
}
```

03-queue_demo.cpp

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    cout << "=== 1. QUEUE (Fila Simples) ===" << endl;
    cout << "Lógica: FIFO (First-In, First-Out) - Primeiro a entrar,
primeiro a sair." << endl;
```

```

// DECLARAÇÃO: queue<tipo> nome;
queue<int> q;

// INSERÇÃO: push(valor) -> insere no FINAL
q.push(10);
q.push(20);
q.push(30); // Fila: [10, 20, 30]

// ACESSO: front() (primeiro) e back() (último)
cout << "Primeiro da fila (front): " << q.front() << endl; // 10
cout << "Último da fila (back): " << q.back() << endl; // 30

// REMOÇÃO: pop() -> remove do INÍCIO (o front)
q.pop(); // Remove o 10
cout << "Novo front após pop: " << q.front() << endl; // 20

// TAMANHO
cout << "Tamanho: " << q.size() << endl;

// Nota: queue NÃO tem acesso por índice (q[0] dá erro).
cout << endl;

cout << "=== 2. DEQUE (Double-Ended Queue) ===" << endl;
cout << "Lógica: Híbrido. Insere e remove nas duas pontas.
Permite acesso por índice." << endl;

// DECLARAÇÃO: deque<tipo> nome;
deque<int> dq;

// INSERÇÃO: push_back() e push_front()
dq.push_back(10); // [10]
dq.push_back(20); // [10, 20]
dq.push_front(5); // [5, 10, 20]
dq.push_front(1); // [1, 5, 10, 20]

// ACESSO: front(), back() e POR ÍNDICE []
cout << "Front: " << dq.front() << endl;
cout << "Back: " << dq.back() << endl;
cout << "Elemento no índice 1 (dq[1]): " << dq[1] << endl; // 0
deque permite isso!

// REMOÇÃO: pop_back() e pop_front()
dq.pop_front(); // Remove o 1. Fica [5, 10, 20]
dq.pop_back(); // Remove o 20. Fica [5, 10]

cout << "Tamanho final: " << dq.size() << endl;
cout << endl;

cout << "=== 3. PRIORITY_QUEUE (Fila de Prioridade) ===" << endl;
;
cout << "Lógica: O elemento de 'maior prioridade' está sempre no
topo." << endl;

// --- TIPO A: MAX-HEAP (Padrão) ---
// O maior número sai primeiro.
// DECLARAÇÃO: priority_queue<tipo> nome;
priority_queue<int> pq_max;

pq_max.push(10);
pq_max.push(50);
pq_max.push(5);

// ACESSO: top() (ATENÇÃO: não é front, é top!)
cout << "Maior elemento (top): " << pq_max.top() << endl; // 50

// REMOÇÃO: pop() -> remove o topo (o maior)
pq_max.pop();
cout << "Novo maior (top): " << pq_max.top() << endl; // 10

// --- TIPO B: MIN-HEAP (Invertida) ---
// O menor número sai primeiro.
// DECLARAÇÃO: priority_queue<tipo, vector<tipo>, greater<tipo>>
nome;
cout << "\n--- Min-Heap (Menor sai primeiro) ---" << endl;
priority_queue<int, vector<int>, greater<int>> pq_min;

pq_min.push(10);
pq_min.push(50);
pq_min.push(5);

cout << "Menor elemento (top): " << pq_min.top() << endl; // 5

pq_min.pop(); // Remove o 5
cout << "Novo menor (top): " << pq_min.top() << endl; // 10

// Nota: priority_queue NÃO tem acesso por índice e nem
iteradores.
// Você só enxerga o topo.

return 0;

```

```

}

```

04-map_demo.cpp

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    cout << "--- Demonstração map ---" << endl;
    cout << "(Pares Chave-Valor, chaves ÚNICAS, ordenado por chave)"
    << endl;

    map<string, int> idades;

    // --- 1. Inserção ---
    // Método 1: insert() com pair
    idades.insert(pair<string, int>("Alice", 30));
    // Método 2: insert() com C++11 (make_pair)
    idades.insert(make_pair("Bob", 25));
    // Método 3: insert() com C++11 (lista de inicialização)
    idades.insert({"Charlie", 35});
    // Método 4: Operador [] (o mais fácil para inserir/atualizar)
    idades["David"] = 28;

    // Tentando inserir chave duplicada (será ignorado)
    idades.insert({"Alice", 99}); // Não vai mudar, "Alice" já
    existe

    // Atualizando valor com []
    idades["Alice"] = 31; // Agora "Alice" tem 31

    // --- 2. Iteração ---
    // Os elementos são iterados em ORDEM DE CHAVE (alfabética aqui)
    cout << "\n--- Iteração (Ordenada) ---" << endl;
    for (const auto& par : idades) {
        // 'par' é um pair<const string, int>
        cout << "Chave: " << par.first << ", Valor: " << par.second
    << endl;
    }

    // --- 3. Busca e Acesso ---
    cout << "\n--- Busca e Acesso ---" << endl;
    // Acesso com []
    cout << "Idade do Bob: " << idades["Bob"] << endl;

    // CUIDADO: Se a chave não existe, o operador [] CRIA um novo
    elemento!
    cout << "Idade do 'Frank' (não existe): " << idades["Frank"] <<
    endl;
    cout << "Tamanho do map agora: " << idades.size() << " (Frank
    foi criado com valor 0)" << endl;

    // Método seguro de busca: find()
    string nome_busca = "David";
    auto it = idades.find(nome_busca); // 'it' é um iterador

    if (it != idades.end()) {
        // .end() é o "local" *após* o último elemento (significa
        "não encontrado")
        cout << nome_busca << " foi encontrado! Idade: " << it->seco
nd << endl;
    } else {
        cout << nome_busca << " não foi encontrado." << endl;
    }

    // --- 4. Remoção ---
    idades.erase("Frank"); // Remove pela chave
    cout << "Tamanho após remover Frank: " << idades.size() << endl;

    //
    -----

    cout << "\n\n--- Demonstração multimap ---" << endl;
    cout << "(Permite chaves DUPLICADAS, ordenado por chave)" << endl;

    multimap<string, string> telefones;
    telefones.insert({"Casa", "1234-5678"});
    telefones.insert({"Casa", "9999-8888"}); // Chave duplicada
    (permitido)
    telefones.insert({"Trabalho", "3333-4444"});

    // Nota: multimap NÃO tem o operador []

    cout << "\n--- Iterando multimap ---" << endl;
    for (const auto& par : telefones) {
        cout << "Local: " << par.first << ", Tel: " << par.second <<
        endl;
    }

    // Buscando todos os valores de uma chave

```

```

    cout << "\n--- Telefones de 'Casa' ---" << endl;
    auto range = telefones.equal_range("Casa");
    // range.first = iterador para o primeiro "Casa"
    // range.second = iterador para DEPOIS do último "Casa"
    for (auto it_tel = range.first; it_tel != range.second; ++it_tel)
    {
        cout << it_tel->second << endl;
    }

    return 0;
}

```

```

auto Proximo = next(Ub);
auto Anterior = prev(Ub);

// 1. Pegando o MENOR (Primeiro elemento)
int menor = *multi_numeros.begin();
// 2. Pegando o MAIOR (último elemento)
int maior = *multi_numeros.rbegin();

return 0;
}

```

05-set_demo.cpp

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    cout << "---- Demonstração set ----" << endl;
    cout << "(Valores ÚNICOS, ordenados)" << endl;

    set<int> numeros;
    numeros.insert(10);
    numeros.insert(5);
    numeros.insert(20);
    numeros.insert(10); // Duplicado: será IGNORADO

    // --- 1. Iteração ---
    // Os elementos são iterados em ORDEM (crescente por padrão)
    cout << "\n--- Iteração (Ordenada) ---" << endl;
    for (int n : numeros) {
        cout << n << " ";
    }
    cout << endl; // Saída: 5 10 20

    // --- 2. Busca (find e count) ---
    cout << "\n--- Busca ---" << endl;

    // find(): retorna um iterador para o elemento ou .end()
    auto it = numeros.find(10);
    if (it != numeros.end()) {
        cout << "O número 10 foi encontrado." << endl;
    } else {
        cout << "O número 10 NÃO foi encontrado." << endl;
    }

    // count(): retorna 1 se o elemento existe, 0 caso contrário (em set)
    if (numeros.count(5)) {
        cout << "O número 5 existe (via count)." << endl;
    }

    // --- 3. Remoção ---
    numeros.erase(10); // Remove o valor 10
    cout << "Após remover 10: ";
    for (int n : numeros) { cout << n << " "; }
    cout << endl;

    // -----

    cout << "\n\n--- Demonstração multiset ----" << endl;
    cout << "(Permite valores DUPLICADOS, ordenados)" << endl;

    multiset<int> multi_numeros;
    multi_numeros.insert(10);
    multi_numeros.insert(5);
    multi_numeros.insert(10); // Duplicado: PERMITIDO
    multi_numeros.insert(10); // Duplicado: PERMITIDO
    multi_numeros.insert(20);

    cout << "\n--- Iteração (Ordenada) ---" << endl;
    for (int n : multi_numeros) {
        cout << n << " ";
    }
    cout << endl; // Saída: 5 10 10 10 20

    // count() agora é muito útil!
    cout << "Quantidade de '10' no multiset: " << multi_numeros.count(10) << endl;

    // CUIDADO: erase(valor) remove TODAS as ocorrências do valor
    multi_numeros.erase(10);
    cout << "Após erase(10): ";
    for (int n : multi_numeros) { cout << n << " "; }
    cout << endl; // Saída: 5 20

    // DA PRA FAZER UPPER_BOUND e LOWER_BOUND no set e no multiset
    auto Ub = multi_numeros.upper_bound(4);
    auto Lb = multi_numeros.lower_bound(8);
}

```

06-adapters_demo.cpp

```

#include <bits/stdc++.h>
using namespace std;

int main() {

    // --- 1. stack (Pilha: LIFO) ---
    cout << "---- Demonstração stack (LIFO) ----" << endl;
    stack<int> pilha;

    pilha.push(10); // Topo -> [10]
    pilha.push(20); // Topo -> [20, 10]
    pilha.push(30); // Topo -> [30, 20, 10]

    cout << "Elemento no topo (top): " << pilha.top() << endl; // 30

    pilha.pop(); // Remove o topo (30)
    cout << "Novo elemento no topo: " << pilha.top() << endl; // 20

    cout << "Tamanho da pilha: " << pilha.size() << endl;

    cout << "Esvaziando a pilha: ";
    while (!pilha.empty()) {
        cout << pilha.top() << " "; // 20, depois 10
        pilha.pop();
    }
    cout << endl;

    // --- 2. queue (Fila: FIFO) ---
    cout << "\n--- Demonstração queue (FIFO) ----" << endl;
    queue<int> fila;

    fila.push(10); // Fim -> [10] <- Início
    fila.push(20); // Fim -> [20, 10] <- Início
    fila.push(30); // Fim -> [30, 20, 10] <- Início

    cout << "Elemento no início (front): " << fila.front() << endl;
    // 10
    cout << "Elemento no fim (back): " << fila.back() << endl; // 30

    fila.pop(); // Remove do início (10)
    cout << "Novo elemento no início: " << fila.front() << endl; // 20

    cout << "Esvaziando a fila: ";
    while (!fila.empty()) {
        cout << fila.front() << " "; // 20, depois 30
        fila.pop();
    }
    cout << endl;

    // --- 3. priority_queue (Fila de Prioridade) ---
    cout << "\n--- Demonstração priority_queue ----" << endl;

    // Por padrão, é uma Max-Heap (o MAIOR elemento fica no topo)
    priority_queue<int> pq_max;
    pq_max.push(10);
    pq_max.push(50);
    pq_max.push(5);
    pq_max.push(20);

    cout << "Topo da Max-Heap: " << pq_max.top() << endl; // 50

    cout << "Esvaziando a Max-Heap (ordem decrescente): ";
    while (!pq_max.empty()) {
        cout << pq_max.top() << " "; // 50, 20, 10, 5
        pq_max.pop();
    }
    cout << endl;

    // Criando uma Min-Heap (o MENOR elemento fica no topo)
    // Sintaxe: priority_queue<Tipo, Contêiner, Comparador>
    priority_queue<int, vector<int>, greater<int>> pq_min;

    pq_min.push(10);
    pq_min.push(50);
    pq_min.push(5);
    pq_min.push(20);
}

```

```

cout << "Topo da Min-Heap: " << pq_min.top() << endl; // 5

cout << "Esvaziando a Min-Heap (ordem crescente): ";
while (!pq_min.empty()) {
    cout << pq_min.top() << " "; // 5, 10, 20, 50
    pq_min.pop();
}
cout << endl;

return 0;
}

```

07-utilities_demo.cpp

```

#include <bits/stdc++.h>
using namespace std;

int main() {

    // --- 1. pair (Agrupa 2 valores) ---
    cout << "--- Demonstração pair ---" << endl;

    // Criação
    pair<int, string> p1(1, "Alice");
    pair<int, string> p2 = make_pair(2, "Bob");
    pair<int, string> p3 = {3, "Charlie"}; // C++11

    // Acesso
    cout << "P1: ID=" << p1.first << ", Nome=" << p1.second << endl;

    p1.first = 10; // Modificando
    cout << "P1 modificado: ID=" << p1.first << ", Nome=" << p1.seco
nd << endl;

    // Desempacotamento (C++17 - Structured Bindings)
    // Este é o jeito moderno de acessar os valores!
    auto [id_p3, nome_p3] = p3;
    cout << "P3 (desempacotado): ID=" << id_p3 << ", Nome=" << nome_
p3 << endl;

    // --- 2. tuple (Agrupa N valores) ---
    cout << "\n--- Demonstração tuple ---" << endl;

    // Criação (Nome, ID, Média)
    tuple<string, int, double> t1("David", 4, 9.5);
    auto t2 = make_tuple("Eve", 5, 8.2); // Tipo inferido

    // Acesso (clássico, usando get<index>)
    cout << "T1: Nome=" << get<0>(t1)
        << ", ID=" << get<1>(t1)
        << ", Média=" << get<2>(t1) << endl;

    get<2>(t1) = 9.8; // Modificando
    cout << "T1 modificado: Média=" << get<2>(t1) << endl;

    // Desempacotamento (C++17 - Structured Bindings)
    // Muito mais limpo que get!
    auto [nome_t2, id_t2, media_t2] = t2;
    cout << "T2 (desempacotado): Nome=" << nome_t2
        << ", ID=" << id_t2
        << ", Média=" << media_t2 << endl;

    // UNIQUE
    vector<int> v = {1,2,2,3,3,3,2,4,4};
    sort(v.begin(), v.end());
    v.erase(unique(v.begin(), v.end()), v.end());

    return 0;
}

```

08-algorithms_demo.cpp

```

#include <bits/stdc++.h>
using namespace std;

// --- ESTRUTURA PARA O EXEMPLO ---
struct Aluno {
    string nome;
    int nota;
};

// --- COMPARADOR CLÁSSICO (Função) ---
bool compararPorNota(const Aluno& a, const Aluno& b) {
    return a.nota < b.nota;
}

// Helper para imprimir
void printAlunos(const string& titulo, const vector<Aluno>& v) {
    cout << titulo << endl;
    for (const auto& a : v) {

```

```

        cout << " { " << a.nome << ", " << a.nota << " }" << endl;
    }
}

int main() {
    cout << "=== 1. SORT vs STABLE_SORT ===" << endl;

    vector<Aluno> turma = {
        {"Ana", 10}, {"Zeca", 5}, {"Bia", 10}, {"Carlos", 8}, {"Xuxa", 5}
    };

    // STABLE SORT: Garante que se as notas forem iguais (Ana e
    Bia),
    // a ordem original entre elas é mantida.
    vector<Aluno> v_stable = turma;
    stable_sort(v_stable.begin(), v_stable.end(), compararPorNota);
    printAlunos("Estabilidade garantida (stable_sort):", v_stable);

    cout << "\n=== 2. COMPARADORES CUSTOMIZADOS (Básico) ===" << end
l;
    vector<int> nums = {1, 5, 2, 8, 3};

    // Lambda Simples (int explícito)
    sort(nums.begin(), nums.end(), [](int a, int b) {
        // Pares primeiro
        if (a % 2 == 0 && b % 2 != 0) return true;
        if (a % 2 != 0 && b % 2 == 0) return false;
        return a < b;
    });

    cout << "Pares primeiro: ";
    for(int n : nums) cout << n << " ";
    cout << endl;

    cout << "\n=== 3. COMPARADORES COM AUTO (C++14) - SUA ESCOLHA
===> << endl;
    // Esta é a forma mais moderna e flexível!

    vector<pair<int, int>> arr = {{10, 5}, {5, 8}, {10, 2}, {2, 9}
};

    // FORMA A: Criando a lambda e guardando em variável
    // 'const auto &' é eficiente e genérico (funciona pra qualquer
    tipo com .first)
    auto f = [] (const auto &a, const auto &b) {
        return a.first < b.first;
    };

    stable_sort(arr.begin(), arr.end(), f);

    cout << "Ordenado pelo First (usando auto f): ";
    for(auto p : arr) cout << "{" << p.first << ", " << p.second << "
} ";
    cout << endl;

    // FORMA B: Lambda Inline (Direto na função)
    // Muito comum em competições para economizar linhas
    // Exemplo: Ordenar pelo SECOND decrescente
    sort(arr.begin(), arr.end(), [](const auto &a, const auto &b) {
        return a.second > b.second;
    });

    cout << "Ordenado pelo Second Decrescente (inline): ";
    for(auto p : arr) cout << "{" << p.first << ", " << p.second << "
} ";
    cout << endl;

    cout << "\n=== 4. OUTROS ALGORITMOS ÚTEIS ===" << endl;
    vector<int> v = {2, 4, 6, 8, 10};

    // Binary Search, Lower Bound, Upper Bound
    // Nota: Todos exigem vetor ordenado
    cout << "Tem 6? " << (binary_search(v.begin(), v.end(), 6) ? "Si
m" : "Nao") << endl;
    cout << "Lower bound 5 (>=): " << *lower_bound(v.begin(), v.end(
), 5) << endl; // 6
    cout << "Upper bound 6 (>): " << *upper_bound(v.begin(), v.end(
), 6) << endl; // 8

    // Min e Max
    cout << "Max: " << *max_element(v.begin(), v.end()) << endl;

    return 0;
}

```

09-loops_demo.cpp

```

#include <bits/stdc++.h>
using namespace std;
#define all(a) (a).begin(),(a).end()

```

```

int main() {
    cout << "---- Demonstração de Laços 'for' ----" << endl;
    vector<int> v = {10, 20, 30, 40, 50};

    // --- 1. Laço 'for' Clássico (baseado em índice) ---
    // Bom para quando você precisa do ÍNDICE (i)
    cout << "\n--- 1. Laço por Índice ----" << endl;
    for (int i = 0; i < v.size(); ++i) {
        cout << "v[" << i << "] = " << v[i] << endl;
    }

    // --- 2. Laço 'for' baseado em Iteradores (STL clássico) ---
    // Funciona com TODOS os contêineres STL (incluindo map, set)
    cout << "\n--- 2. Laço por Iterador ----" << endl;
    for (auto it = v.begin(); it != v.end(); ++it) {
        // 'it' é um iterador (um "ponteiro" para o elemento)
        // Use *it para acessar o valor
        cout << "Valor: " << *it << endl;
    }

    // --- 3. Laço 'for' baseado em Intervalo (Range-based) - C++11
    ---
    // O mais moderno, limpo e seguro. Prefira este!
    cout << "\n--- 3. Laço por Intervalo (Range-based C++11) ----" <<
endl;

    // 3a. Lendo os valores (cópia ou const referência)
    cout << "Lendo valores: ";
    // 'const auto&' é o mais eficiente: 'const' (não vou mudar)
    // '&' (referência, evita copiar o objeto)
    for (const auto& elemento : v) {
        cout << elemento << " ";
        // elemento = 100; // Erro! É constante
    }
    cout << endl;

    // 3b. Modificando os valores (usando referência não-const)
    cout << "Modificando valores (dobrando)..." << endl;
    // 'auto&' (referência): 'elemento' é um *apelido* para o item
original no vetor
    for (auto& elemento : v) {
        elemento = elemento * 2;
    }

    cout << "Vetor após modificação: ";
    for (const auto& elemento : v) {
        cout << elemento << " ";
    }
    cout << endl;

    // Exemplo com map (onde o índice não existe)
    map<string, int> idades = {"Alice", 30}, {"Bob", 25};
    cout << "\n--- Range-based com Map ----" << endl;
    // 'par' será um pair
    for (const auto& par : idades) {
        cout << "Chave: " << par.first << ", Valor: " << par.second
<< endl;
    }

    // NEXT_PERMUTATION (Lembrar da ordem inicial ser o menor valor
possível)

    vector<int> Seq = {1,2,3,4,5,6,7,8};
    do {
        for(int x : Seq) {
            cout << x << "\n";
        }
    } while(next_permutation(all(Seq)));

    return 0;
}

```

10-advanced_techniques_demo.cpp

```

#include <bits/stdc++.h>
using namespace std;

// --- 1. Soma de Prefixo (Prefix Sum) ---
// Técnica O(N) para pré-calcular, permitindo consultas
// de soma de intervalo (i, j) em O(1).
void demoPrefixSum() {
    cout << "---- 1. Demonstração Soma de Prefixo (Prefix Sum) ----" <
< endl;
    vector<int> v = {2, 8, 3, 5, 7, 4, 1, 9};
    int n = v.size();

    // Cria um vetor prefix_sum de tamanho n
    // prefix_sum[i] = soma de v[0]..v[i]
    vector<long long> prefix_sum(n);

```

```

// Método 1: Usando partial_sum (do <numeric>)
partial_sum(v.begin(), v.end(), prefix_sum.begin());

/* // Método 2: Manual (mais comum em competições)
prefix_sum[0] = v[0];
for (int i = 1; i < n; ++i) {
    prefix_sum[i] = prefix_sum[i-1] + v[i];
}
*/

cout << "Vetor original: ";
for(int x : v) cout << x << " ";
cout << "\nVetor Prefix Sum: ";
for(long long x : prefix_sum) cout << x << " ";
cout << endl;

// --- Consulta de Intervalo O(1) ---
// Queremos a soma do intervalo [i, j] (inclusivo)
int i = 2; // (valor 3)
int j = 5; // (valor 4)
// Soma(i, j) = Soma(0, j) - Soma(0, i-1)

long long soma_intervalo = prefix_sum[j];
if (i > 0) {
    soma_intervalo -= prefix_sum[i-1];
}

// Soma = 3 + 5 + 7 + 4 = 19
cout << "Soma do intervalo [" << i << ", " << j << "] = " << som
a_intervalo << endl;

// Suffix Sum (Soma de Sufixo) é a mesma lógica, mas do final
para o começo.
}

// --- 2. Mediana Móvel (Running Median) ---
// Técnica para encontrar a mediana de um fluxo de números
// de forma eficiente (O(log N) por inserção).
// Usa duas PQs:
// 1. lowers (Max-Heap): Armazena a metade INFERIOR dos números
// 2. highs (Min-Heap): Armazena a metade SUPERIOR dos números
class RunningMedian {
private:
    priority_queue<int> lowers; // Max-Heap
    priority_queue<int, vector<int>, greater<int>> highs; //
Min-Heap

    // Garante que as heaps estejam balanceadas (tamanhos diferem em
no máximo 1)
    void rebalance() {
        if (lowers.size() > highs.size() + 1) {
            highs.push(lowers.top());
            lowers.pop();
        } else if (highs.size() > lowers.size() + 1) {
            lowers.push(highs.top());
            highs.pop();
        }
    }

public:
    void addNumber(int num) {
        // Adiciona o novo número
        if (lowers.empty() || num < lowers.top()) {
            lowers.push(num);
        } else {
            highs.push(num);
        }

        // Rebalanceia as heaps
        rebalance();
    }

    double getMedian() {
        if (lowers.empty() && highs.empty()) {
            return 0.0;
        }

        // Se tamanhos iguais (nº par de elementos), mediana é a
média dos topos
        if (lowers.size() == highs.size()) {
            return (lowers.top() + highs.top()) / 2.0;
        }

        // Se tamanhos diferentes (nº ímpar), mediana é o topo da
heap maior
        else if (lowers.size() > highs.size()) {
            return lowers.top();
        } else {
            return highs.top();
        }
    }
};

```

```

void demoRunningMedian() {
    cout << "\n--- 2. Demonstração Mediana Móvel (Running Median)
    ---" << endl;
    RunningMedian medianFinder;
    vector<int> fluxo = {5, 2, 10, 8, 1, 12};

    cout << "Adicionando números e calculando a mediana:" << endl;
    for (int num : fluxo) {
        medianFinder.addNumber(num);
        cout << "Adicionado: " << setw(2) << num
            << " | Mediana atual: " << medianFinder.getMedian() << endl;
    }
}

int main() {
    // Chama a primeira demo
    demoPrefixSum();

    // Chama a segunda demo
    demoRunningMedian();

    return 0;
}

```

11-buscas_binarias_demo.cpp

```

#include <bits/stdc++.h>
using namespace std;

// --- FUNÇÕES AUXILIARES (Para usar dentro da main) ---

// 1. Busca Padrão (Encontra o valor exato)
int binarySearchPadrao(const vector<int>& arr, int target) {
    int low = 0;
    int high = arr.size() - 1;

    while (low <= high) {
        // CÁLCULO SEGURO DO MEIO (Evita Overflow de int)
        // mid = low + (distancia / 2)
        int mid = low + (high - low) / 2;

        if (arr[mid] == target) return mid; // Achou!

        if (arr[mid] < target)
            low = mid + 1; // Está na direita -> descarta esquerda
        else
            high = mid - 1; // Está na esquerda -> descarta direita
    }
    return -1; // Não achou
}

// 2. Lower Bound (Primeiro Verdadeiro)
// Exemplo: Achar o PRIMEIRO número que é >= target
int buscaPrimeiroTrue(const vector<int>& arr, int target) {
    int low = 0, high = arr.size() - 1;
    int ans = -1; // Guarda a melhor resposta encontrada

    while (low <= high) {
        int mid = low + (high - low) / 2;

        // Condição: É maior ou igual? (Verdadeiro)
        if (arr[mid] >= target) {
            ans = mid; // Opa, serve! Salva o índice.
            high = mid - 1; // Mas sou ganancioso: tento achar um
            // índice MENOR à esquerda.
        } else {
            low = mid + 1; // Falso. Preciso aumentar o valor.
        }
    }
    return ans;
}

// 3. Last True (Último Verdadeiro)
// Exemplo: Achar o ÚLTIMO número que é <= target
int buscaUltimoTrue(const vector<int>& arr, int target) {
    int low = 0, high = arr.size() - 1;
    int ans = -1;

    while (low <= high) {
        int mid = low + (high - low) / 2;

        // Condição: É menor ou igual? (Verdadeiro)
        if (arr[mid] <= target) {
            ans = mid; // Serve! Salva o índice.
            low = mid + 1; // Mas sou ganancioso: tento achar um
            // índice MAIOR à direita.
        } else {
            high = mid - 1; // Falso. O valor já passou do limite.
        }
    }
}

```

```

    }
    return ans;
}

// -----

int main() {
    // Vetor ordenado para os exemplos (REGRA DE OURO: Tem que estar
    // ordenado)
    // Índices:      0  1  2  3  4  5  6  7  8  9
    vector<int> v = {2,  5,  8, 12, 16, 23, 38, 56, 72, 91};

    cout << "Vetor: {2, 5, 8, 12, 16, 23, 38, 56, 72, 91}" << endl;
    cout << "Tamanho: " << v.size() << endl << endl;

    cout << "=== 1. BUSCA BINARIA PADRAO (Exact Match) ===" << endl;
    cout << "Lógica: Divide ao meio. Se igual, retorna. Se maior,
    vai pra dir. Se menor, vai pra esq." << endl;
    cout << "Fórmula do Mid: mid = low + (high - low) / 2;
    (Anti-Overflow)" << endl;

    int alvo = 23;
    int idx = binarySearchPadrao(v, alvo);

    if (idx != -1)
        cout << "Alvo " << alvo << " encontrado no índice: " << idx
        << endl;
    else
        cout << "Alvo não encontrado." << endl;

    cout << endl;

    cout << "=== 2. BUSCA NA RESPOSTA: TIPO 'LOWER BOUND' (Primeiro
    True) ===" << endl;
    cout << "Cenário: [F, F, F, T, T, T]. Quero o PRIMEIRO T." << endl;

    cout << "Lógica: Se check(mid) é True -> Salva 'ans = mid' e
    tenta ESQUERDA (high = mid - 1)." << endl;

    // Exemplo: Primeiro número >= 20
    // No vetor: 16 (F), 23 (T), 38 (T)... O primeiro T é o 23
    // (índice 5).
    int limiteMin = 20;
    int idxPrimeiro = buscaPrimeiroTrue(v, limiteMin);

    cout << "Primeiro elemento >= " << limiteMin << " está no
    índice: " << idxPrimeiro;
    cout << " (Valor: " << v[idxPrimeiro] << ")" << endl;
    cout << endl;

    cout << "=== 3. BUSCA NA RESPOSTA: TIPO 'LAST TRUE' (Último
    True) ===" << endl;
    cout << "Cenário: [T, T, T, F, F, F]. Quero o ÚLTIMO T." << endl;

    cout << "Lógica: Se check(mid) é True -> Salva 'ans = mid' e
    tenta DIREITA (low = mid + 1)." << endl;

    // Exemplo: Último número <= 40
    // No vetor: 38 (T), 56 (F)... O último T é o 38 (índice 6).
    int limiteMax = 40;
    int idxUltimo = buscaUltimoTrue(v, limiteMax);

    cout << "Último elemento <= " << limiteMax << " está no índice:
    " << idxUltimo;
    cout << " (Valor: " << v[idxUltimo] << ")" << endl;
    cout << endl;

    cout << "=== 4. USANDO STL (Bibliotecas do C++) ===" << endl;
    cout << "Dica: Em competições/trabalho, use as funções prontas
    se for vetor simples." << endl;

    // binary_search -> Retorna bool (só diz se existe ou não)
    bool existe = binary_search(v.begin(), v.end(), 56);
    cout << "binary_search(56): " << (existe ? "True" : "False") << endl;

    // lower_bound -> Retorna ITERADOR para o primeiro elemento >=
    // valor
    auto it = lower_bound(v.begin(), v.end(), 20);
    cout << "lower_bound(20): aponta para " << *it << " (Índice: " <<
    (it - v.begin()) << ")" << endl;

    // upper_bound -> Retorna ITERADOR para o primeiro elemento
    // ESTRITAMENTE MAIOR que valor
    // Útil para saber onde inserir algo sem quebrar a ordem
    auto it2 = upper_bound(v.begin(), v.end(), 23); // Busca > 23
    cout << "upper_bound(23): aponta para " << *it2 << " (Índice: "
    << (it2 - v.begin()) << ")" << endl;

    return 0;
}

```

12-MinElement_MaxElement.cpp

```
#include <bits/stdc++.h>
using namespace std;
#define all(a) (a).begin(),(a).end()

int main() {
    vector<int> v = {3,2,7,1,8};

    int Min = *min_element(all(v));

    int Max = *max_element(all(v));
}
```

13-list_demo.cpp

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    // 1. DECLARAÇÃO
    list<string> compras;

    // 2. INSERÇÃO NAS EXTREMIDADES (O(1))
    compras.push_back("Arroz"); // Adiciona no fim
    compras.push_back("Feijao");
    compras.push_front("Cafe"); // Adiciona no início (muito
rápido na list)

    // 3. USO DE ITERADORES (O "Cursor")
    // Como não existe compras[1], usamos iteradores para navegar
    auto it = compras.begin();
    it++; // Agora o cursor aponta para o segundo elemento ("Arroz")

    // 4. INSERÇÃO NO MEIO (O(1) se você já tem o iterador)
    compras.insert(it, "Açúcar"); // Insere ANTES da posição do
iterador

    // 5. REMOÇÃO (O(1))
    compras.pop_front(); // Remove o primeiro ("Cafe")

    // Para remover um elemento específico, usamos o iterador
    // Vamos remover onde o 'it' está apontando ("Arroz")
    compras.erase(it);

    // 6. UTILITÁRIOS ÚTEIS
    cout << "Tamanho da lista: " << compras.size() << endl;

    compras.sort(); // Ordena alfabeticamente

    compras.reverse(); // Inverte a ordem

    // 7. LIMPEZA
    compras.clear();
    if (compras.empty()) {
        cout << "A lista está vazia agora!" << endl;
    }

    return 0;
}
```

02 - BUSCAS

00-BuscaBinaria.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
#define pii pair<int,int>
#define FORi(n) for(int i=0;i<n;i++)
#define FORj(n) for(int j=0;j<n;j++)
#define all(a) (a).begin(),(a).end()
#define printn(v,n) for(int i=0;i<n;i++) cout << v[i] << (i == n-1 ?
"\n" : " ")

bool check(ll x) { return true; }

void solve() {
    ll low = 0, high = 9e18;
    ll ans = -1; // Guarda a melhor resposta encontrada

    while(low <= high) {
        ll mid = low + (high-low) / 2;

        // Condição: É (Verdadeiro)? // ans = mid; no que for
        salvar
        if(check(mid)) { high = mid - 1; ans = mid; } // Salva o
        índice / Tenta achar um índice MENOR à esquerda.
        else { low = mid + 1; } // Falso. Preciso aumentar o valor.
    }
    cout << ans << "\n";
}

int main() {
    ios::sync_with_stdio(false);cin.tie(0);
    solve();
}
```

01-Selecionar-Eventos.cpp

```
#include <bits/stdc++.h>
using namespace std;
#define pii pair<int,int>
#define FORi(n) for(int i=0;i<n;i++)
#define all(a) (a).begin(),(a).end()

void solve() {
    int n; cin>>n;

    vector<pii> Momentos(n);
    int start,end;
    FORi(n) {
        cin>>start>>end;
        Momentos[i] = {start,end};
    }

    // SORT PELO QUE TERMINA PRIMEIRO
    auto f = [] (const auto &a, const auto &b) {
        if(a.second != b.second) { return a.second < b.second; }
        else { return a.first >= b.first; }
    };
    stable_sort(all(Momentos), f);

    // VERIFICA SE O PROXIMO DA FILA COMEÇA DEPOIS DO FIM DO ATUAL,
    // SE SIM ESCOLHE ELE E TESTA O PROXIMO...
    pii prev = {-1,-1};
    int qtd_max = 0;
    FORi(n) {
        pii atual = Momentos[i];
        if(atual.first >= prev.second) { prev = atual; qtd_max++; }
    }
    cout << qtd_max << "\n";
}

int main() {
    ios::sync_with_stdio(false);cin.tie(0);
    int t; cin>>t; while(t--) { solve(); }
}

// OBS: Ficar atento que tem questões que não consideram que se pode
// assistir um evento que começa em X depois de um que terminou em X
// Nesses casos troca o >= por >
```

02-Selecionar-Eventos_K-pessoas.cpp

```
#include <bits/stdc++.h>
using namespace std;
#define pii pair<int,int>
#define FORi(n) for(int i=0;i<n;i++)
#define all(a) (a).begin(),(a).end()
```

```
void solve() {
    int n,k;
    cin>>n>>k;

    vector<pii> Momentos(n);
    int start,end;
    FORi(n) {
        cin>>start>>end;
        Momentos[i] = {start,end};
    }

    auto f = [] (const auto &a, const auto &b) {
        if(a.second != b.second) { return a.second < b.second; }
        else { return a.first >= b.first; }
    };
    stable_sort(all(Momentos), f);

    multiset<int> Hora_Livre;
    FORi(k) { Hora_Livre.insert(-1); }
    int qtd_max = 0;
    FORi(n) {
        pii atual = Momentos[i];
        // Primeira pessoa Indisponivel
        auto Ub = Hora_Livre.upper_bound(atual.first);

        // Se tem alguém Disponível
        if(Ub != Hora_Livre.begin()) {
            // Primeira pessoa Disponível
            Ub = prev(Ub);
            Hora_Livre.erase(Ub);
            Hora_Livre.insert(atual.second);
            qtd_max++;
        }
    }
    cout << qtd_max << "\n";
}

int main() {
    ios::sync_with_stdio(false);cin.tie(0);
    solve();
}
```

03-Stack_Monotonica.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
#define FORi(n) for(int i=0;i<n;i++)
#define all(a) (a).begin(),(a).end()

void solve(){
    int n;
    cin>>n;

    vector<int> K(n);
    FORi(n) { cin>>K[i]; }

    ll max_area = 1LL * n * *min_element(all(K));
    stack<int> Stack_Mono;
    K.push_back(0);

    FORi(n+1) {
        while(!Stack_Mono.empty() && K[i] < K[Stack_Mono.top()]) {
            ll h = K[Stack_Mono.top()];
            Stack_Mono.pop();

            if(Stack_Mono.empty()) {
                max_area = max(max_area, h*i);
            }
            else {
                int largura = i - Stack_Mono.top() - 1;
                max_area = max(max_area, h*largura);
            }
        }
        Stack_Mono.push(i);
    }
    cout << max_area << "\n";
}

int main(){
    ios::sync_with_stdio(false);cin.tie(0);
    solve();
}
```

04-Psum2D.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
```

```

vector<vector<ll>> Psum2D(vector<vector<ll>> &A) {
    int n = A.size();
    int m = A[0].size();

    vector<vector<ll>> Pre(n, vector<ll>(m, 0));

    for(int i = 0; i < n; i++) {
        for(int j = 0; j < m; j++) {

            Pre[i][j] = A[i][j];

            if(i > 0) {
                Pre[i][j] += Pre[i-1][j];
            }
            if(j > 0) {
                Pre[i][j] += Pre[i][j-1];
            }
            if(i > 0 && j > 0) {
                Pre[i][j] -= Pre[i-1][j-1];
            }
        }
    }
    return Pre;
}

ll SubmatSum(vector<vector<ll>> &Pre, int r1, int c1, int r2, int c2)
{
    ll soma = Pre[r2][c2];

    if(r1 > 0) {
        soma -= Pre[r1-1][c2];
    }
    if(c1 > 0) {
        soma -= Pre[r2][c1-1];
    }
    if(r1 > 0 && c1 > 0) {
        soma += Pre[r1-1][c1-1];
    }
    return soma;
}

```

05-SparseTable.cpp

```

// Sparse Table - RMQ (static)
//
// build - O(n log n)
// query - O(1)

// Funciona: min, max, gcd, bitwise AND, bitwise OR.
// Essa técnica de consulta em O(1) só funciona para operações
// onde elementos repetidos não alteram o resultado final join(x,x)
// = x

struct Sparse {
    int n;
    vector<int> lg;
    vector<vector<int>> tab;

    int join(int a, int b) {
        return min(a,b);
    }

    Sparse(const vector<int>& V){
        n = V.size();

        lg.assign(n+1, 0);
        for(int i=2;i<=n;i++) lg[i] = lg[i/2] + 1;

        int K = lg[n]+1;
        tab.assign(K, vector<int>(n));

        tab[0]=V;

        for(int j=1;j<K;j++) {
            for(int i=0;i+(1<<j)-1<n;i++) {
                tab[j][i] = join( tab[j-1][i], tab[j-1][i+(1<<(j-1))] );
            }
        }

        int query(int l, int r) {
            int j = lg[r-l+1];
            return join( tab[j][l], tab[j][r-(1<<j)+1] );
        }
    };

    /* USO:

    vector<int> v(n);

```

```

for(int i=0;i<n;i++) cin>>v[i];

Sparse sp(v);
cout<<sp.query(l,r)<<"\n"; // se o problema der l,r
1-indexado: query(l-1,r-1)

*/

```

06-SparseTable2D.cpp

```

// Sparse Table 2D - RMQ (static)
//
// build - O(n * m * log(n) * log(m))
// query - O(1)

struct Sparse2D {
    int n, m;
    vector<int> lg;
    vector<vector<vector<vector<int>>>> tab;

    int join(int a, int b) {
        return min(a, b);
    }

    Sparse2D(const vector<vector<int>>& M) {
        if (M.empty() || M[0].empty()) return;
        n = M.size();
        m = M[0].size();

        // Pré-computa os logaritmos até o maior lado
        int max_len = max(n, m);
        lg.assign(max_len + 1, 0);
        for(int i = 2; i <= max_len; i++) lg[i] = lg[i/2] + 1;

        int KR = lg[n] + 1;
        int KC = lg[m] + 1;

        // tab[kr][kc][r][c]
        tab.assign(KR, vector<vector<vector<int>>>)(
            KC, vector<vector<int>>(n, vector<int>(m))
        ));

        // Caso base (kr = 0, kc = 0)
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < m; j++) {
                tab[0][0][i][j] = M[i][j];
            }
        }

        // Construção DP
        for (int kr = 0; kr < KR; kr++) {
            for (int kc = 0; kc < KC; kc++) {
                if (kr == 0 && kc == 0) continue;

                for (int i = 0; i + (1 << kr) - 1 < n; i++) {
                    for (int j = 0; j + (1 << kc) - 1 < m; j++) {
                        if (kr == 0) {
                            // Junta na horizontal
                            tab[kr][kc][i][j] = join(
                                tab[kr][kc-1][i][j],
                                tab[kr][kc-1][i][j + (1 << (kc-1))]
                            );
                        } else {
                            // Junta na vertical (podendo juntar
                            // matrizes já computadas na horizontal)
                            tab[kr][kc][i][j] = join(
                                tab[kr-1][kc][i][j],
                                tab[kr-1][kc][i + (1 << (kr-1))][j]
                            );
                        }
                    }
                }
            }
        }

        int query(int r1, int c1, int r2, int c2) {
            int kr = lg[r2 - r1 + 1];
            int kc = lg[c2 - c1 + 1];

            // Sobrepe os 4 retângulos que cobrem a query completa
            int top_left = tab[kr][kc][r1][c1];
            int top_right = tab[kr][kc][r1][c2 - (1 << kc) + 1];
            int bottom_left = tab[kr][kc][r2 - (1 << kr) + 1][c1];
            int bottom_right = tab[kr][kc][r2 - (1 << kr) + 1][c2 - (1 <
            < kc) + 1];

            return join(join(top_left, top_right), join(bottom_left, bot
            tom_right));
        }
    };

    // USO:
};

```

```

/* USO:

    int n, m;
    cin >> n >> m;
    vector<vector<int>> mat(n, vector<int>(m));
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            cin >> mat[i][j];
        }
    }

    Sparse2D sp(mat);

    // Consultando o retângulo do canto superior esquerdo (r1, c1)
    // até o inferior direito (r2, c2)
    cout << sp.query(r1, c1, r2, c2) << "\n"; // Se for 1-indexado,
    faça r1-1, c1-1, etc.

*/

```

03- GRAFOS_1

01-MatrizAdj-SemPeso.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// N até mais ou menos 5e3
const int MAXSZ = 1e3 + 5;
bool g[MAXSZ][MAXSZ];

void addEdge(int u, int v) {
    g[u][v] = 1;
    // g[v][u] = 1;
}

void removeEdge(int u, int v) {
    g[u][v] = 0;
    // g[v][u] = 0;
}
```

02-MatrizAdj-ComPeso.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// N até mais ou menos 5e3
const int MAXSZ = 1e3 + 5;
const int INF = 2e9;
int g[MAXSZ][MAXSZ];
int n,m;

void init(){
    for(int i=0;i<n;i++){
        for(int j=0;j<m;j++){
            g[i][j] = INF;
        }
    }
}

void addEdge(int u, int v, int w){
    g[u][v] = w;
    // g[v][u] = w;
}

void removeEdge(int u, int v, int w){
    g[u][v] = INF;
    // g[v][u] = INF;
}
```

03-ListaAdj-SemPeso.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

const int MAXSZ = 1e3 + 5;
int n,m;

vector<int> adj[MAXSZ];

void addEdge(int u, int v){
    adj[u].push_back(v);
    // adj[v].push_back(u);
}
```

04-ListaAdj-ComPeso.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
#define pii pair<int,int>

const int MAXSZ = 1e3 + 5;
int n,m;

vector<pii> adj[MAXSZ];

void addEdge(int u, int v, int w){
    adj[u].push_back({v,w});
    // adj[v].push_back({u,w});
}
```

05-PecorrerGrid.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

const int MAXN = 2e5 + 7;
int n,m;
const int dx[] = {1, 0, 0, -1};
const int dy[] = {0, 1, -1, 0};
char grid[MAXN][MAXN];

bool valid(int x, int y){
    return x>=0 && x<n && y>=0 && y<m && grid[x][y]!='#';
}

void rec(int x, int y){
    grid[x][y] = '#'; // Se for usar grid depois usar uma matriz de
    visitas (bool)
    for(int d=0;d<4;d++){
        int ax = x + dx[d], ay = y + dy[d];
        if(!valid(ax,ay)) { continue; }
        rec(ax,ay);
    }
}
```

06-DFS.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

const int MAXN = 2e5 + 7;
vector<int> adj[MAXN];
bool vis[MAXN]; // false para todos

void dfs(int u){
    vis[u] = true;
    for(int v : adj[u]){
        if(!vis[v]) { dfs(v); }
    }
}
```

07-BFS.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

const int MAXN = 2e5 + 7;
vector<int> adj[MAXN];
int dist[MAXN];

void bfs(int b){
    memset(dist, -1, sizeof(dist));
    dist[b] = 0;
    queue<int> q;
    q.push(b);

    while(!q.empty()) {
        int u = q.front(); q.pop();
        for(int v : adj[u]) {
            if(dist[v] == -1) {
                dist[v] = dist[u] + 1;
                q.push(v);
            }
        }
    }
}
```

08-BFS-Grid.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
#define pii pair<int,int>

const int MAXN = 1001;

int n,m;
char grid[MAXN][MAXN];
bool vis[MAXN][MAXN];

const int dx[] = {1, 0, 0, -1};
const int dy[] = {0, 1, -1, 0};

bool inRange(int x, int y){
    return x>=0 && x<n && y>=0 && y<m;
}
```

```

void bfs(){
    queue<pii> q;
    memset(vis, false, sizeof(vis));
    vis[0][0] = true;
    q.push({0,0});

    while(!q.empty()){
        auto [x,y] = q.front(); q.pop();
        for(int i=0;i<4;i++) {
            int ax = x + dx[i], ay = y + dy[i];
            if(inRange(ax,ay) && !vis[ax][ay] && grid[ax][ay]!='#')
            {
                vis[ax][ay] = true;
                q.push({ax,ay});
            }
        }
    }
}

```

09-Bicoloridade.cpp

```

#include <bits/stdc++.h>
using namespace std;
using ll = long long;

vector<vector<int>>> adj;
vector<bool> vis, color;

// Retorna True se for Bipartível
bool dfs(int v){
    vis[v] = true;
    for(int u : adj[v]){
        if(!vis[u]) {
            color[u] = !color[v];
            if(!dfs(u)) { return false; }
        }
        else if(color[v] == color[u]) { return false; }
    }
    return true;
}

```

10-Toposort.cpp

```

#include <bits/stdc++.h>
using namespace std;
using ll = long long;
#define pii pair<int,int>

const int MAXN = 2e5 + 7;
vector<int> adj[MAXN];
int deg[MAXN];

vector<int> toposort(int n) {
    memset(deg, 0, sizeof(deg));

    for(int u=1;u<=n;u++) {
        for(int v : adj[u]) { deg[v]++; }
    }

    queue<int> q;
    for(int i=1;i<=n;i++) {
        if(deg[i] == 0) { q.push(i); }
    }

    vector<int> ord;
    while(!q.empty()) {
        int u = q.front(); q.pop();
        ord.push_back(u);
        for(int v : adj[u]) {
            deg[v]--;
            if(deg[v] == 0) { q.push(v); }
        }
    }

    // se ord.size() < n, não há ordenação possível!
    return ord;
}

// edges redundantes

vector<int> toposort_old(vector<pii>& edges, int& n){
    memset(deg, 0, sizeof(deg));

    for(auto [u,v] : edges) {
        deg[v]++;
    }

    queue<int> q;
    for(int i=1;i<=n;i++) {

```

```

        if(deg[i] == 0) { q.push(i); }
    }

    vector<int> ord;
    while(!q.empty()) {
        int u = q.front(); q.pop();
        ord.push_back(u);
        for(int v : adj[u]) {
            deg[v]--;
            if(deg[v] == 0) { q.push(v); }
        }
    }

    // se ord.size() < n, não há ordenação possível!
    return ord;
}

```

11-BFS-Multisource.cpp

```

#include <bits/stdc++.h>
using namespace std;
using ll = long long;

const int MAXN = 2e5 + 7;
vector<int> adj[MAXN];
int dist[MAXN];

void MultiSourceBFS(const vector<int>& Vertices) {
    memset(dist, -1, sizeof(dist));
    queue<int> q;
    for(int V : Vertices) {
        dist[V] = 0;
        q.push(V);
    }

    while(!q.empty()) {
        int u = q.front(); q.pop();
        for(int v : adj[u]) {
            if(dist[v] == -1) {
                dist[v] = dist[u] + 1;
                q.push(v);
            }
        }
    }
}

// OBS:

// Problema: Dado um grafo com K vértices marcados.
// Queremos saber, para cada vértice, a menor distância até algum
// vértice marcado.

// Solução: No início da BFS, defina a distância de cada vértice
// marcado como 0.

// Inicie a fila da BFS já contendo todos os vértices marcados.

// Intuição: As camadas da BFS vão se formar
// ao redor de cada vértice marcado, simultaneamente.

// Assim, qualquer vértice será alcançado primeiro
// justamente pelo vértice marcado que está mais próximo dele.

// Observação: Cada vértice continua sendo visitado apenas uma única
// vez.

```

12-Find-Cycles.cpp

```

#include <bits/stdc++.h>
using namespace std;
using ll = long long;

const int MAXN = 2e5 + 7;
vector<int> adj[MAXN];
bool vis[MAXN];
vector<int> ciclo;

int dfs(int atual, int anterior) {
    if(vis[atual]) { return atual; } // ciclo encontrado: retorna
    início do ciclo
    vis[atual] = true;

    for(int nxt : adj[atual]) {
        if(nxt == anterior) { continue; }
        int fim = dfs(nxt,atual);

        if(fim == -2) { return -2; } // ciclo já fechado e
        propagando término
        if(fim != -1) {
            ciclo.push_back(atual);

```

```

        if(atual == fim) { return -2; }    // ciclo fechado
completamente
        else { return fim; }    // ainda estamos subindo no
ciclo
    }
}
return -1;    // não encontrou ciclo pelo caminho
}

// OBS:

// Para encontrar ciclos em um grafo não direcionado,
// podemos usar uma busca em profundidade (DFS).

// Começamos a percorrer o grafo visitando vértices livremente.

// Se, em algum momento, chegarmos a um vértice que já foi visitado
// e não é o vértice anterior (de onde viemos),
// isso indica que existe um ciclo.

// Nesse caso, podemos voltar pelo caminho percorrido
// até reencontrar o mesmo vértice
// e registrar todos os nós que fazem parte do ciclo.

```

13-Diametro-Arvore.cpp

```

// Dado uma árvore, o seu diâmetro é a maior distância entre dois
vértices
// Podem existir diferentes pares que representam o diâmetro,
// mas a distância entre cada par é a maior possível.

// Primeiro, escolhe-se qualquer nó e realiza-se uma busca (BFS ou
DFS)
// para encontrar o nó mais distante, chamado de A.

// Em seguida, faz-se uma nova busca começando em A
// para encontrar o nó mais distante dele, chamado de B.

// A distância entre A e B é o diâmetro da árvore,
// isto é, o comprimento do maior caminho simples entre dois nós.

// Obs: Em uma árvore existe exatamente um único caminho entre
quaisquer dois vértices.

// Portanto, a distância pode ser calculada tanto com BFS quanto com
DFS.

```

14-CompConexos-FloodFill.cpp

```

#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int n;
const int MAXN = 2e5 + 7;
vector<int> adj[MAXN];
int componente[MAXN];

void dfs(int u, int comp) {
    componente[u] = comp;
    for(int v : adj[u]) {
        if(componente[v] == -1) {
            dfs(v, comp);
        }
    }
}

void init(){
    memset(componente, -1, sizeof(componente));

    int num = 0;
    for(int i=1; i<=n; i++) {
        if(componente[i] == -1) {
            num++;
            dfs(i, num);
        }
    }
}

// Um componente conexo é um subconjunto de vértices do grafo
// original em que todos conseguem chegar uns aos outros.

// Podemos encontrar o componente conexo
// de um vértice usando apenas uma DFS ou BFS.

// Em matrizes, essa mesma ideia recebe o nome de Flood Fill.

// A intuição é que funciona como "pintar um desenho dentro das
linhas".

```

04- GRAFOS_2

01-Dijkstra.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
#define pli pair<ll,int>

const int MAXN = 1e5 + 10;

vector<ll> djikstra(vector<vector<pli>> &adj, int ini=1) {
    vector<ll> dist(MAXN, LLONG_MAX);
    priority_queue<pli, vector<pli>, greater<pli>> pq;

    // custo e inicial
    pq.push({0, ini});
    dist[ini] = 0;

    while(!pq.empty()) {
        auto [total, atual] = pq.top(); pq.pop();

        if(total <= dist[atual]) {
            for(auto [peso, proximo] : adj[atual]) {
                if(dist[proximo] > dist[atual] + peso) {
                    dist[proximo] = dist[atual] + peso;
                    pq.push({dist[proximo], proximo});
                }
            }
        }
    }
    return dist;
}

int main() {
    int n = 1e5 + 10;
    vector<ll> dist(n+1, LLONG_MAX);
    vector<vector<pair<int,ll>>> adj(n+1);
    priority_queue<pli, vector<pli>, greater<pli>> pq;

    // custo e inicial
    pq.push({0,1});
    dist[1] = 0;

    while(!pq.empty()) {
        auto [d,u] = pq.top(); pq.pop();

        if(d <= dist[u]) {
            for(auto [v,w] : adj[u]) {
                if(dist[v] > dist[u] + w) {
                    dist[v] = dist[u] + w;
                    pq.push({dist[v], v});
                }
            }
        }
    }
}
```

02-Dijkstra_Sem_PQ.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

const int n = 1e5 + 10;
vector<ll> dist(n + 1, LLONG_MAX);
vector<bool> vis(n + 1, false);
vector<vector<tuple<ll, ll>>> adj(n + 1);

int main() {
    // Exemplo: setar a distância inicial do vértice 1
    dist[1] = 0;

    // Dijkstra O(n^2)
    for (int i = 1; i <= n; i++) {
        int v = -1;
        ll mn = LLONG_MAX;

        // encontra vértice com menor distância não visitado
        for (int j = 1; j <= n; j++) {
            if (!vis[j] && (dist[j] < mn)) {
                mn = dist[j];
                v = j;
            }
        }

        if (v == -1) break;
        vis[v] = true;
```

```
        // relaxar arestas do vértice v
        for (auto [u, w] : adj[v]) {
            if (dist[u] > dist[v] + w) {
                dist[u] = dist[v] + w;
            }
        }
    }

    return 0;
}
```

03-Floyd-Warshall.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

const int n = 300 + 10;

int main() {
    vector<vector<ll>> dist(n+1, vector<ll>(n+1));
    vector<tuple<ll,int,int>> arestas;

    // Inicializa Dist com LLONG_MAX
    for(int i=1;i<=n;i++) { for(int j=1;j<=n;j++) { dist[i][j] = LLONG_MAX; } }
    //dist[i][j] deve ser 0 se (i==j)
    for(int i=1;i<=n;i++) { dist[i][i] = 0; }
    //Cuidado com múltiplas arestas (pegue o mínimo)
    for(auto& [cost,u,v] : arestas) { dist[u][v] = dist[v][u] = min(dist[u][v], cost); }

    for(int k=1;k<=n;k++) {
        for(int u=1;u<=n;u++) {
            for(int v=1;v<=n;v++){
                if((dist[u][k] != LLONG_MAX) && (dist[k][v] != LLONG_MAX)) {
                    dist[u][v] = min(dist[u][v], dist[u][k] + dist[k][v]);
                }
            }
        }
    }
}
```

04-Bellman-Ford.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

const int n = 2500 + 1;
int parent[n];

int main() {
    vector<tuple<int,int,ll>> edges;
    vector<ll> dist(n+1, 0);
    // Bellman-Ford
    for(int rep=0;rep<n-1;rep++) {
        for(auto [u,v,w] : edges) {
            if(dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                // V foi alcançado por U
                parent[v] = u;
            }
        }
    }

    // Ciclo Negativo?
    bool cicloNegativo = false;
    for(auto [u,v,w] : edges) {
        if(dist[v] > dist[u] + w) {
            cicloNegativo = true;
            parent[v] = u;
        }
    }
}
```

05-Modelagem.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// Modelagem é transformar o problema em outro grafo que represente os estados do problema e permita a utilização dos algoritmos clássicos.
```

```

const int n = 2500 + 1;
int parent[n];

int main() {

    vector<vector<ll>> dist(n+1, vector<ll>(n+1, LLONG_MAX));
    vector<vector<tuple<ll, ll>>> adj(n+1);
    priority_queue<tuple<ll, ll, ll>,
        vector<tuple<ll, ll, ll>>,
        greater<tuple<ll, ll, ll>>> pq;

    while(!pq.empty()){
        auto [d,v,c] = pq.top(); pq.pop();

        if(d <= dist[v][c]){
            for(auto [u,w] : adj[v]) {
                if(dist[u][c] > dist[v][c] + w) {
                    dist[u][c] = dist[v][c] + w;
                    pq.push({dist[u][c], u, c});
                }
            }
            if(c < 10){
                if(dist[u][c+1] > dist[v][c] + (w/2)) {
                    dist[u][c+1] = dist[v][c] + (w/2);
                    pq.push({dist[u][c+1], u, c+1});
                }
            }
        }
    }
}

```

06-DSU.cpp

```

#include <bits/stdc++.h>
using namespace std;
using ll = long long;

const int MAXN = 1e5 + 7;
int pai[MAXN], sz[MAXN];

int find(int u) {
    if(u==pai[u]) { return u; }
    return pai[u] = find(pai[u]);
}

void join(int u, int v) {
    u = find(u);
    v = find(v);
    if(u == v) { return; }
    if(sz[u] < sz[v]) { swap(u,v); }
    pai[v] = u;
    sz[u] += sz[v];
}

int main() {
    int n;
    // Inicializa os Grupos
    for(int u=1;u<=n;u++) {
        pai[u] = u;
        sz[u] = 1;
    }
}

```

07-MST-Kruskal.cpp

```

#include <bits/stdc++.h>
using namespace std;
using ll = long long;
#define all(a) (a).begin(),(a).end()

// Para encontrar uma MST podemos usar o algoritmo guloso de
// kruskal.
// Iterando pelas arestas ordenadas pelo custo, se a aresta atual
// conecta componentes distintos, adicionamos ela à resposta. Isto é,
// não forma um ciclo na árvore!
// Para verificar se um ciclo será formado, usamos DSU!

// N-1 ARESTAS = ARVORE
// MST = ARVORE COM MENOR CUSTO

const int MAXN = 1e5 + 7;
int pai[MAXN], sz[MAXN];

int find(int u){
    if(u==pai[u]) { return u; }
    return pai[u] = find(pai[u]);
}

void join(int u, int v){

```

```

    u = find(u);
    v = find(v);
    if(u == v) { return; }
    if(sz[u] < sz[v]) { swap(u,v); }
    pai[v] = u;
    sz[u] += sz[v];
}

int main() {
    int n,m;

    vector<tuple<ll,int,int>> arestas;

    // Ordena por Custo das Estradas
    sort(all(arestas));
    ll totalCost = 0;

    // Inicializa os Grupos
    for(int u=1;u<=n;u++) {
        pai[u] = u;
        sz[u] = 1;
    }

    int arestas_usadas = 0;
    for(auto [cost,u,v] : arestas) {
        if(find(u) != find(v)){
            join(u,v);
            totalCost += cost;
            arestas_usadas++;
        }
    }

    if(arestas_usadas == n-1) { cout << totalCost << "\n"; }
    else { cout << "IMPOSSIBLE\n"; }
}

```

Infos.txt

Dijkstra
 $O(N+M \log N)$
 Caminho de um para todos. Pesos positivos.

Bellman-Ford
 $O(N * M)$
 Caminho de um para todos. Pesos podem ser negativos.

Floyd-Warshall
 $O(N^3)$
 Caminho de todos para todos.

05-DP_1

01-LIS.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
#define FORi(n) for(int i=0;i<n;i++)
#define all(a) (a).begin(),(a).end()

// 0(n^2);
// da pra fazer com 0(n*log n)

int solve() {
    int n = 9;
    vector<int> A = {10, 9, 2, 5, 3, 7, 101, 9, 20};

    int ans = 0;
    vector<int> dp(n);
    for(int j=0;j<n;j++) {
        dp[j] = 1;
        for(int i=j-1;i>=0;i--) {
            if(A[i] < A[j]) {
                dp[j] = max(dp[j], dp[i] + 1);
            }
        }
        ans = max(ans, dp[j]);
    }
    return ans;
}

// 0 que é o LIS (Longest Increasing Subsequence)?
// Dado um array, queremos achar o tamanho da maior subseqüência
// estritamente crescente.

// Importante: subseqüência != subarray (podemos pular elementos).

// 0(n*log n)
// SÓ SERVE PRA GARANTIR O TAMANHO, E NAO A SUBSEQ EM SI

int solve2() {
    int n = 9;
    vector<int> a = {10, 9, 2, 5, 3, 7, 101, 9, 20};

    vector<int> lis;
    for(int x : a) {
        // lower_bound faz a busca binária:
        // encontra o primeiro elemento >= x
        auto it = lower_bound(lis.begin(), lis.end(), x);

        // if - Se não encontrou ninguém >= x, x é o maior de todos
        // else - Se encontrou, substitui aquele valor por x
        // else - Isso mantém a "cauda" a menor possível
        if(it == lis.end()) { lis.push_back(x); }
        else { *it = x; }
    }

    return lis.size();
}

// LIS de consecutivos
// 0(n*log n)
int lis_consecutive() {
    int n = 9;
    vector<int> A = {10, 9, 2, 5, 3, 7, 101, 9, 20};

    map<int,int> dp;
    int ans = 0;
    int prev = -1; // print seq

    FORi(n) {
        int x = A[i];
        dp[x] = dp[x-1] + 1;
        if(dp[x] > ans) {
            ans = dp[x];
            prev = x; // print seq
        }
    }

    // Printando a maior Seq
    vector<int> res;
    int count = 0;
    for(int i=n-1;i>=0;i--) {
        if(A[i]==prev) {
            res.push_back(i+1);
            prev--;
            count++;
        }
        if(count>=ans) { break; }
    }
}
```

```
reverse(all(res));
}
```

02-LCS.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

vector<vector<int>>> dp;
string a,b;

int best(int i, int j) {
    if(i < 0 or j < 0) { return 0; }
    if(dp[i][j] != -1) { return dp[i][j]; }

    dp[i][j] = 0;
    if(a[i] == b[j]) { dp[i][j] = 1 + best(i-1, j-1); }
    else { dp[i][j] = max(best(i-1, j), best(i, j-1)); }
    return dp[i][j];
}

int solve() {
    a = "acbd";
    int n = 4;
    b = "abed";
    int m = 4;

    dp = vector<vector<int>>>(n, vector<int>(m, -1));

    return best(n-1, m-1);
}

// 0 que é o LCS (Longest Common Subsequence)?
// LCS = maior subseqüência comum entre duas strings, preservando a
// ordem, mas não necessariamente contigua.
```

03-SubsetSum.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

bool solve() {
    int t = 1;
    vector<int> a = {3, 4, 7, 2};

    vector<bool> dp(t + 1);
    dp[0] = true;

    for (int x : a) {
        for (int i = t - x; i >= 0; --i) {
            if (dp[i]) {
                dp[i + x] = true;
            }
        }
    }

    return dp[t];
}

// 0 que é o Subset Sum?
// É o problema de decidir se existe algum subconjunto de um
// conjunto de
// inteiros que some exatamente um valor-alvo T.
```

04-Knapsack.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int solve() {
    int n = 4, W = 7;
    vector<int> w = {3, 4, 2, 6};
    vector<int> v = {4, 5, 3, 7};

    vector<int> knapsack(W + 1);
    for (int i=0;i<n;i++) {
        for (int k=W-w[i];k>=0;k--) {
            knapsack[k + w[i]] = max(knapsack[k + w[i]], knapsack[k]
+ v[i]);
        }
    }
    return knapsack[W];
}

// Knapsack
// 0 que é o Knapsack?
```

```
// Você tem: uma mochila com capacidade W,
// n itens, cada um com: peso w[i] e valor v[i]

// Pergunta: qual o maior valor que dá pra colocar na mochila sem
passar do peso?
// Cada item pode ser pego 1 vez ou não pego
```

05-Reconstrucao.txt

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

const int MAXN = 1e5 + 7;
vector<int> adj[MAXN];

int solve() {
    // Como atualizar as tabelas

    // dp[i] - Guarda o melhor valor para aquela posição
    // rec[i] - Guarda de qual posição anterior veio o valor

    int n = 5;

    vector<long long> dp(n, LLONG_MIN);
    vector<int> rec(n, -1);

    int start = 0;
    dp[start] = 0;

    for (int u = 0; u < n; u++) {
        for (int v : adj[u]) {
            long long cand = dp[u] + w[u][v];

            if (cand > dp[v]) {
                dp[v] = cand;
                rec[v] = u;
            }
        }
    }

    // Como reconstruir a resposta

    vector<int> caminho;

    int s = estado_final;

    while (s != estado_base) {
        caminho.push_back(s);
        s = rec[s];
    }

    caminho.push_back(estado_base);
    reverse(caminho.begin(), caminho.end());
}
```

06-ValorEsperado.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ld = long double;
#define FORi(n) for(int i=0;i<n;i++)

void solve() {
    int n,x; cin>>n>>x;

    vector<ld> P(n);
    FORi(n) {
        ld p; cin>>p;
        p /= 100;
        P[i] = p;
    }

    vector<vector<ld>> dp_Pr(n+1, vector<ld>(n+1, 0.0));
    // Probabilidade de um Pacote com i Cartas, ter exatamente j
    cartas raras
    for(int i=0;i<=n;i++) {
        if(i==0) { dp_Pr[0][0] = 1; }
        else { dp_Pr[0][i] = 0; }
    }
    for(int i=1;i<=n;i++) {
        for(int j=0;j<=n;j++) {
            if(j > 0) { dp_Pr[i][j] = dp_Pr[i-1][j-1] * P[i-1] + dp_
Pr[i-1][j] * (-P[i-1] + 1); }
            else { dp_Pr[i][j] = dp_Pr[i-1][j] * (-P[i-1] + 1); }
        }
    }

    vector<ld> dp(x+1, 0.0);
```

```
// Numero esperado de pacotes que eu preciso abrir, pra ter X
raras, dado que eu tenho i raras
for(int i=x-1;i>=0;i--) {
    dp[i] = 1;
    for(int j=1;j<=n;j++) {
        if(i+j <= x) { dp[i] += dp[i+j] * dp_Pr[n][j]; }
        else { break; }
    }
    dp[i] /= (-dp_Pr[n][0] + 1);
}
cout << dp[0] << "\n";
}

int main() {
    ios::sync_with_stdio(false);cin.tie(0);
    cout << fixed << setprecision(9);

    solve();
}
```

06-ValorEsperado2.cpp

```
#include <bits/stdc++.h>
using namespace std;
#define FORi(n) for(int i=0;i<n;i++)

const int MAXN = 2026;

void solve() {
    int n,p; cin>>n>>p;

    // prob de pegar uma figurinha nova
    // prob[i][j], dado que eu tenho i figurinhas de j totais
    vector<vector<double>> prob(MAXN+1, vector<double>(MAXN+1, 0.0));

    for(int j=1;j<=MAXN;j++) {
        for(int i=0;i<j;i++) {
            double pb = j-i; pb /= j;
            prob[i][j] = pb;
        }
    }

    double j_novo = p; j_novo /= 100;

    // quantidade esperada de dias pra completar
    // dp[i][j], dado que eu tenho i figurinhas de j totais
    vector<vector<double>> dp(MAXN+1, vector<double>(MAXN+1, 0.0));
    for(int j=MAXN;j>=n;j--) {
        for(int i=j-1;i>=0;i--) {
            dp[i][j] = 1;

            double novo = j_novo;
            if(j == MAXN) { novo = 0.0; }

            // figurinha nova, sem jogador novo
            dp[i][j] += dp[i+1][j] * prob[i][j] * (-novo + 1);

            // figurinha nova, com jogador novo
            if(j < MAXN) { dp[i][j] += dp[i+1][j+1] * prob[i][j+1] *
novo; }

            // figurinha repetida, com jogador novo
            if(j < MAXN) { dp[i][j] += dp[i][j+1] * (-prob[i][j+1] +
1) * novo; }

            // divide pela 1-prob de manter o estado
            double w = (-prob[i][j] + 1); // repetida
            w *= (-novo + 1); // sem novo
            dp[i][j] /= (-w + 1);
        }
    }
    cout << dp[0][n] << "\n";
}

int main() {
    ios::sync_with_stdio(false);cin.tie(0);
    cout << fixed << setprecision(12);

    solve();
}
```

06-DP_2

01-Operacoes.txt

Bit Shifting

```
(1 << n) == 2^n
```

o 1 foi empurrado 4 vezes pra esquerda

```
// para esquerda: multiplica por 2
// para direita: divide por 2
```

Operações básicas

```
// AND: x & y
// OR: x | y
// XOR: x ^ y
```

```
// Ligar i-ésimo bit:
(n |= (1 << i))
```

```
// Testar se o i-ésimo bit está ligado:
(n & (1 << i))
```

```
// Flipar(switch) o i-ésimo bit:
(n ^= (1 << i))
```

Tricks

```
// Testar se é potencia de 2:
!(n & (n-1))
```

```
// Contar quantos bits estão ligados:
__builtin_popcount(n)
ou
__builtin_popcountll(n)
```

```
// Bitmask com 'n' 1s:
((1 << n) - 1)
```

```
// LSB:
(n & -n)
```

```
// Gerar todos os Subsets:
for(int mask=0; mask<(1 << n); mask++) {
    for(int idx=0; idx<n; idx++) {
        if(mask & (1 << idx)) {
            //está presente no subset
            // lembrar dos parênteses,
            // o operador '==' tem preferência
        }
    }
}
```

// BASES DIFERENTE

```
int int_pow(int base, int n) {
    if(n==0) { return 1; }
}
```

```
int x = 1;
FORi(n) { x *= base; }
return x;
```

// Valor do Bit

```
int base_bit_value(int value, int bit, int B) {
    int x = (value / int_pow(B, bit));
    return x%B;
}
```

```
// Incrementa o Bit idx na Base3
mask += int_pow(3, idx);
```

```
string pra ll com base customizada
// stoll(s, nullptr, <base:16>);
```

02-DP_Subconjuntos.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
```

```
const ll INF = 1e18; // número arbitrariamente grande
```

```
void solve() {
    int n;
    int BITS = 3;
    string juice;
```

```
vector<ll> dp(1 << BITS, INF);
dp[0] = 0; // não é necessário nenhum custo para obter o
conjunto vazio
```

```
for (int i=0; i<n; i++) {
    int mask = 0;
    for(char vitamin : juice) {
        if(vitamin == 'A') mask |= 1; // liga o primeiro bit
        else if(vitamin == 'B') mask |= 2; // liga o segundo bit
        else if(vitamin == 'C') mask |= 4; // liga o terceiro bit
    }

    int cost;
    for(int j=0; j<(1 << BITS); j++) {
        int x = j | mask; // para cada possível estado, liga os
        bits do mask
        dp[x] = min(dp[x], dp[j] + cost);
    }

    if(dp[7] == INF) { cout << -1 << "\n"; }
    else { cout << dp[7] << "\n"; }
}
```

03-Backtracking_TSP.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

const int INF = 2e9;
int best = INF;
int n = 4;
const int MAXN = 4;
int C[MAXN][MAXN];
vector<int> perm;
int cur_cost = 0;

void backtracking_tsp(int i, int used){
    int cnt = __builtin_popcount(used);
    if(cnt == n){
        best = min(best, cur_cost);
        return;
    }

    for(int j = 0; j < n; j++) if(!(used & (1<<j))){
        // faz
        used ^= (1<<j);
        perm.push_back(j);
        cur_cost += C[i][j];
        backtracking_tsp(j, used);
        // desfaz
        cur_cost -= C[i][j];
        perm.pop_back();
        used ^= (1<<j);
    }
}
```

04-TSP_Recursivo.cpp

```
#include <bits/stdc++.h>
using namespace std;
#define FORi(n) for(int i=0; i<n; i++)
#define FORj(n) for(int j=0; j<n; j++)

// MENOR SOMA DOS PARES DE DISTANCIAS

int n;
const int MOD = 1e9 + 7;
const int MAXN = 21;
int grid[MAXN][MAXN];
int dp[MAXN][1<<MAXN];
const int INF = 2e9;

int DP(int i, int mask) {
    if(mask == (1<<n) - 1) { return dp[i][0]; }

    int& ans = dp[i][mask];
    if(ans != -1) { return ans; }

    ans = INF;
    FORj(n) {
        // Se o Bit j já foi usado (continue)
        if(mask & (1<<j)) { continue; }
        ans = min(ans, grid[i][j] + DP(j, mask ^ (1<<j)));
    }
    return ans;
}
```

```

void solve(){
    // Inicializacao da dp
    FORi(n) {
        FORj(1<<n) {
            dp[i][j] = -1;
        }
    }
}

```

05-TSP_Iterativa.cpp

```

#include <bits/stdc++.h>
using namespace std;
using ll = long long;

const int MAXN = 20;
const int INF = 2e9;
int dp[1<<MAXN][MAXN];
int grid[MAXN][MAXN];

void solve() {
    int n = 4;
    for(int mask=0;mask<(1<<n);mask++) {
        for(int i=0;i<n;i++) { dp[mask][i] = INF; }
    }

    dp[1][0] = 0;
    for(int mask = 1; mask < (1<<n); mask++) {
        for(int i = 0; i < n; i++) if(mask & (1<<i)) {
            for(int j = 0; j < n; j++) {
                if(mask & (1<<j)) { continue; }
                dp[mask ^ (1<<j)][j] = min(dp[mask ^ (1<<j)][j], dp[
mask][i] + grid[i][j]);
            }
        }
    }

    int best = INF;
    for(int i = 1; i < n; i++) {
        best = min(best, dp[(1<<n) - 1][i] + grid[i][0]);
    }
}

```

07-TEORIA DOS NUMEROS

00-Matrix.cpp

```
template<typename T> struct Matrix {
    vector<vector<T>>> mat;
    // Linhas x Colunas
    int n,m;

    Matrix(int N, int M=0) : n(N), m(M?M:N){ mat.assign(n, vector<T>(m, 0)); }

    friend Matrix operator* (const Matrix &a, const Matrix &b){
        assert(a.m == b.n);
        Matrix ans(a.n, b.m);
        for(int i=0; i<a.n; i++)
            for(int j=0; j<b.m; j++)
                for(int k=0; k<a.m; k++)
                    ans.mat[i][j] += a.mat[i][k] * b.mat[k][j];
        return ans;
    }
};
```

00-Matrix_Mod.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

const int MOD = 1e9 + 7;

ll mod_add(ll a, ll b) { return ((a % MOD) + (b % MOD)) % MOD; }
ll mod_mul(ll a, ll b) { return ((a % MOD) * (b % MOD)) % MOD; }

template<typename T> struct Matrix {
    vector<vector<T>>> mat;
    // Linhas x Colunas
    int n,m;

    Matrix(int N, int M=0) : n(N), m(M?M:N){ mat.assign(n, vector<T>(m, 0)); }

    friend Matrix operator* (const Matrix &a, const Matrix &b){
        assert(a.m == b.n);
        Matrix ans(a.n, b.m);
        for(int i=0; i<a.n; i++)
            for(int j=0; j<b.m; j++)
                for(int k=0; k<a.m; k++)
                    ans.mat[i][j] = mod_add(ans.mat[i][j], mod_mul(a.mat[i][k], b.mat[k][j]));
        return ans;
    }
};
```

01-Divisores.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// complexidade temporal: O(sqrt(n))
// complexidade espacial: O(qntdivisores(n))

vector<ll> getdivisors(ll n) {
    vector<ll> divisors;
    for(ll i=1; i*i<=n; i++) {
        if(n%i == 0) {
            divisors.push_back(i);
            if(i != n/i) { divisors.push_back(n/i); }
        }
    }
    return divisors;
}
```

02-Fatoracao_Primeira.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
#define pli pair<ll,int>

// complexidade temporal: O(sqrt(n))
// complexidade espacial: O(qnt_fatores_primos(n))

vector<pli> getprimes(ll n) {
    vector<pli> primes;
    for(ll i=2; i*i<=n; i++) {
        int cnt = 0;
```

```
        while(n%i == 0) {
            n /= i;
            cnt++;
        }

        if(cnt > 0) { primes.push_back({i, cnt}); }
    }

    if(n > 1) { primes.push_back({n, 1}); }
    return primes;
}
```

03-Crivo.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// Complexidade Temporal: O(Nlog(log(N)))
// Complexidade Espacial: O(N)

const int LIM = 1e6 + 5;
bool isPrime[LIM];

vector<int> sieve() {
    memset(isPrime, 1, sizeof(isPrime));
    isPrime[0] = isPrime[1] = false;

    for(ll i=2; i*i<LIM; i++) {
        if(isPrime[i]) {
            for(ll j=i*i; j<LIM; j+=i) { isPrime[j] = false; }
        }
    }

    vector<int> primes;
    for(int i=2; i<LIM; i++) {
        if(isPrime[i]) { primes.push_back(i); }
    }

    return primes;
}

// MARCA PRA TODOS OS VALORES DE 1 ATÉ LIM
// QUAIS OS FATORES PRIMOS CONTEM NA FATORAÇÃO DELE

const int LIM = 2e5 + 5;
vector<bool> vis(LIM+1, false);
vector<vector<int>> primos(LIM);

void sieve2() {
    for(int i=2; i<=LIM; i++) {
        if(vis[i]) { continue; }
        vis[i] = true;
        for(int j=i; j<=LIM; j+=i) {
            vis[j] = true;
            primos[j].push_back(i);
        }
    }
}

const int LIM = 1e6 + 5;
bool isPrime[LIM];

vector<int> sieve3() {
    memset(isPrime, 1, sizeof(isPrime));
    isPrime[0] = isPrime[1] = false;

    vector<int> primes;
    for(int i=2; i<LIM; i++) {
        if(isPrime[i]) {
            primes.push_back(i);
            for(int j=i*i; j<LIM; j+=i) { isPrime[j] = false; }
        }
    }

    return primes;
}
```

04-Crivos_Especiais.txt

```
1. Calcular a soma dos divisores de todos os números no intervalo [1, LIM]

#include <bits/stdc++.h>
using namespace std;

// Complexidade Temporal: O(N log N)
// Complexidade Espacial: O(N)
```

```

const int LIM = 100000; // ajuste conforme necessário
long long sumDivisors[LIM];

void sieve() {
    for (int i = 1; i < LIM; i++) {
        for (int j = i; j < LIM; j += i) {
            sumDivisors[j] += i;
        }
    }
}

int main() {
    sieve();

    // Exemplo de uso:
    for (int i = 1; i <= 10; i++) {
        cout << "Soma dos divisores de " << i << " = " << sumDivisor
s[i] << '\n';
    }

    return 0;
}

//

2. Calcular a quantidade de divisores de todos os números no intervalo [1,LIM]

#include <bits/stdc++.h>
using namespace std;

// Complexidade Temporal: O(N log N)
// Complexidade Espacial: O(N)

const int LIM = 100000; // ajuste conforme necessário
long long numDivisors[LIM];

void sieve() {
    for (int i = 1; i < LIM; i++) {
        for (int j = i; j < LIM; j += i) {
            numDivisors[j]++;
        }
    }
}

int main() {
    sieve();

    // Exemplo de uso:
    for (int i = 1; i <= 10; i++) {
        cout << "Quantidade de divisores de " << i << " = " << numDivis
ors[i] << '\n';
    }

    return 0;
}

```

05-Aritmetica_Modular.cpp

```

#include <bits/stdc++.h>
using namespace std;
using ll = long long;

const int MOD = 1e9 + 7;

ll fexp(ll base, ll pot){
    ll ans = 1;
    while(pot) {
        if(pot & 1) { ans = ans * base % MOD; }
        base = base * base % MOD;
        pot >>= 1;
    }
    return ans;
}

ll mod_add(ll a, ll b) { return ((a % MOD) + (b % MOD)) % MOD; }

ll mod_sub(ll a, ll b) { return (((a % MOD) - (b % MOD)) + MOD) % MOD; }

ll mod_mul(ll a, ll b) { return ((a % MOD) * (b % MOD)) % MOD; }

ll mod_div(ll a, ll b) { return mod_mul(a, fexp(b, MOD-2)); }

```

06-FastExp.cpp

```

#include <bits/stdc++.h>
using namespace std;
using ll = long long;

```

```

const int MOD = 1e9 + 7;

ll fexp(ll base, ll pot) {
    ll ans = 1;
    while(pot) {
        if(pot & 1) { ans = ans * base % MOD; }
        base = base * base % MOD;
        pot >>= 1;
    }
    return ans;
}

```

07-Mat_FastExp.cpp

```

#include <bits/stdc++.h>
using namespace std;
using ll = long long;
#define FORi(n) for(int i=0;i<n;i++)

const int MOD = 1e9 + 7;

ll mod_add(ll a, ll b) { return ((a % MOD) + (b % MOD)) % MOD; }
ll mod_mul(ll a, ll b) { return ((a % MOD) * (b % MOD)) % MOD; }

template<typename T> struct Matrix {
    vector<vector<T>>> mat;
    // Linhas x Colunas
    int n,m;

    Matrix(int N, int M=0) : n(N), m(M?M:N){ mat.assign(n, vector<T>(m, 0)); }

    friend Matrix operator* (const Matrix &a, const Matrix &b){
        assert(a.m == b.n);
        Matrix ans(a.n, b.m);
        for(int i=0;i<a.n;i++)
            for(int j=0;j<b.m;j++)
                for(int k=0;k<a.m;k++)
                    ans.mat[i][j] = mod_add(ans.mat[i][j], mod_mul(a.mat[i][k], b.mat[k][j]));
        return ans;
    }
};

Matrix<ll> mat_fexp(Matrix<ll> base, ll pot){
    // Matriz Identidade
    Matrix<ll> ans(base.n, base.m);
    FORi(base.n) { ans.mat[i][i] = 1; }

    while(pot) {
        if(pot & 1) { ans = ans * base; }
        base = base * base;
        pot >>= 1;
    }
    return ans;
}

```

08-Combinatoria_Modular.cpp

```

#include <bits/stdc++.h>
using namespace std;
using ll = long long;

const int MAXN = 1e3 + 7;
const int MOD = 1e9 + 7;
ll fact[MAXN], invFact[MAXN];

ll fexp(ll base, ll pot) {
    ll ans = 1;
    while(pot) {
        if(pot & 1) { ans = ans * base % MOD; }
        base = base * base % MOD;
        pot >>= 1;
    }
    return ans;
}

// APENAS PRA MOD PRIMO
ll inv(ll x) { return fexp(x, MOD-2); }

// Complexidade O(MAXN)
void precompute() {
    fact[0] = 1;
    for(int i=1;i<MAXN;i++) {
        fact[i] = (fact[i-1] * i) % MOD;
    }

    // Calcula o inverso do último fact
    invFact[MAXN-1] = inv(fact[MAXN-1]);
}

```

```

// Preenche o resto de tras para frente
for(int i=MAXN-2;i>=0;i--) {
    invFact[i] = (invFact[i+1] * (i+1)) % MOD;
}

ll binomial(ll n, ll k) {
    if(k > n || k < 0) { return 0; }
    return ((fact[n] * invFact[k] % MOD) * invFact[n-k]) % MOD;
}

int main(){
    ios::sync_with_stdio(false);cin.tie(0);

    // PRECOMPUTAR NO MAIN
    precompute();
}

```

09-NTT.cpp

```

#include <bits/stdc++.h>
using namespace std;
using ll = long long;

const int MOD = 998244353;
const int G = 3; // Raiz primitiva para o MOD 998244353

ll fexp(ll base, ll pot) {
    ll ans = 1;
    while(pot) {
        if(pot & 1) { ans = ans * base % MOD; }
        base = base * base % MOD;
        pot >>= 1;
    }
    return ans;
}

// Algoritmo NTT (Number Theoretic Transform)
void ntt(vector<ll>& a, bool invert) {
    int n = a.size();

    // Inversão de bits (Bit-reversal permutation)
    for (int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1) j ^= bit;
        j ^= bit;
        if (i < j) swap(a[i], a[j]);
    }

    // Borboleta do NTT (Butterfly operation)
    for (int len = 2; len <= n; len <= 1) {
        ll wlen = fexp(G, (MOD - 1) / len);
        if (invert) wlen = fexp(wlen, MOD - 2); // Inverso modular

        for (int i = 0; i < n; i += len) {
            ll w = 1;
            for (int j = 0; j < len / 2; j++) {
                ll u = a[i + j];
                ll v = (a[i + j + len / 2] * w) % MOD;
                a[i + j] = (u + v < MOD ? u + v : u + v - MOD);
                a[i + j + len / 2] = (u - v >= 0 ? u - v : u - v + MOD);
                w = (w * wlen) % MOD;
            }
        }
    }

    // Se for a NTT Inversa, divide pelo tamanho do vetor N
    if (invert) {
        ll n_inv = fexp(n, MOD - 2);
        for (ll& x : a) {
            x = (x * n_inv) % MOD;
        }
    }
}

// Multiplica dois polinômios usando NTT e limita o tamanho ao N_alvo
vector<ll> multiply(vector<ll> a, vector<ll> b, int max_degree) {
    int n = 1;
    while (n < a.size() + b.size()) n <= 1;

    a.resize(n, 0);
    b.resize(n, 0);

    ntt(a, false);
    ntt(b, false);

    for (int i = 0; i < n; i++) {
        a[i] = (a[i] * b[i]) % MOD;
    }
}

```

```

    ntt(a, true);

    // Corta o polinômio para não carregar graus desnecessários maiores que N
    if (a.size() > max_degree + 1) {
        a.resize(max_degree + 1);
    }
    return a;
}

// Exponenciação Binária de Polinômios: Eleva P(x) à potência D
vector<ll> poly_pow(vector<ll> P, ll D, int max_degree) {
    // Começa com o polinômio identidade: 1 (ou seja, 1 * x^0)
    vector<ll> res = {1};

    while (D > 0) {
        if (D % 2 == 1) {
            res = multiply(res, P, max_degree);
        }
        P = multiply(P, P, max_degree);
        D /= 2;
    }
    return res;
}

int main() {
    // Exemplo do problema:
    // P(x) = x^1 + x^3 + x^4 + x^6
    // Representação por coeficientes: índice é o expoente
    // P[0]=0, P[1]=1 (x^1), P[2]=0, P[3]=1 (x^3), P[4]=1 (x^4), P[5]=0, P[6]=1 (x^6)
    vector<ll> P = {0, 1, 0, 1, 1, 0, 1};

    ll D = 2; // 2 dias
    int N_alvo = 7; // Queremos saber de quantas formas somamos 7 ienes

    // Calcula P(x)^D limitando o polinômio resultante ao grau N_alvo
    vector<ll> resultado = poly_pow(P, D, N_alvo);

    // O coeficiente no índice N_alvo é a nossa resposta
    if (N_alvo < resultado.size()) {
        cout << "Formas de conseguir " << N_alvo << " ienes em " <<
        D << " dias: " << resultado[N_alvo] << endl;
    } else {
        cout << "Formas de conseguir " << N_alvo << " ienes em " <<
        D << " dias: 0" << endl;
    }

    return 0;
}

```

08-RANGEQUERY

01-MergeSort.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
#define all(a) (a).begin(),(a).end()

vector<ll> Merge(vector<ll>& A, vector<ll>& B) {
    vector<ll> C;
    int l = 0, r = 0;
    while(l < A.size() && r < B.size()) {
        if(A[l] <= B[r]) { C.push_back(A[l]); l++; }
        else { C.push_back(B[r]); r++; }
    }
    while(l < A.size()) {
        C.push_back(A[l]); l++;
    }
    while(r < B.size()) {
        C.push_back(B[r]); r++;
    }
    return C;
}

vector<ll> Merge2(vector<ll>& A, vector<ll>& B) {
    vector<ll> C;
    merge(all(A), all(B), back_inserter(C));
    return C;
}

// Tendo os arrays A e B ordenados, queremos construir C ordenado
// possuindo todos elementos de A e B.
// Two pointers! O(n)
```

02-Merge_Inversoes.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

vector<ll> Merge(vector<ll>& A, vector<ll>& B) {
    vector<ll> C;
    int l = 0, r = 0;
    while(l < A.size() && r < B.size()) {
        if(A[l] <= B[r]) { C.push_back(A[l]); l++; }
        else { C.push_back(B[r]); r++; }
    }
    while(l < A.size()) {
        C.push_back(A[l]); l++;
    }
    while(r < B.size()) {
        C.push_back(B[r]); r++;
    }
    return C;
}

// O(N * log N)

void Merge_Inv(int l, int r, ll& inversions, vector<ll>& A) {
    if(l >= r) { return; }

    int m = (l+r) / 2;
    Merge_Inv(l, m, inversions, A);
    Merge_Inv(m+1, r, inversions, A);

    // Os ranges [l,m] e [m+1,r] tao ordenados
    vector<ll> Left;
    for(int i=l; i<=m; i++) { Left.push_back(A[i]); }
    vector<ll> Right;
    for(int i=m+1; i<=r; i++) { Right.push_back(A[i]); }

    int R = 0;
    for(auto x : Left) {
        while(R < Right.size() && Right[R] <= x) { R++; }
        inversions += R;
    }

    vector<ll> C = Merge(Left, Right);
    for(int i=l; i<=r; i++) { A[i] = C[i-l]; }
}

// Dado um array, queremos contar a quantidade de inversões.

// Uma inversão em um array é quando para um par de índices i < j,
// acontece a[i] > a[j]. Exemplo [1,4,5,7,2].

// Array ordenado não decrescentemente possui 0 inversões.
// [1, 2, 3, 5, 6] = 0 inversões
```

```
// Array ordenado decrescentemente possui o máximo de inversões.
// [6, 5, 3, 2, 1] = 10 inversões
```

03-SegmentTree.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// NÓ CUSTOMIZADO
struct Node { int freq = 0; ll soma = 0; };

template<typename T> struct SegTree {
    vector<T> Seg;
    int N;
    // Valor Neutro (Trocar de acordo com o Tipo da Seg)
    T NEUTRO = 2e9;

    SegTree(int n) {
        N = 1; while(N < n) { N *= 2; }
        Seg.assign(N*2, NEUTRO);
    }

    SegTree(vector<T> &Lista) {
        N = 1; while(N < Lista.size()) { N *= 2; }
        Seg.assign(N*2, NEUTRO); build(1, 0, N-1, Lista);
    }

    // União da Seg (Trocar a Operação de acordo com o Tipo da Seg)
    T join(T lv, T rv) { return min(lv, rv); }

    T query(int no, int l, int r, int a, int b){
        if(b < l || r < a) { return NEUTRO; }
        if(a <= l && r <= b) { return Seg[no]; }
        int m = (l+r)/2, e = no*2, d = e+1;

        return join(query(e, l, m, a, b), query(d, m+1, r, a, b));
    }

    void update(int no, int l, int r, int pos, T v){
        if(pos < l || r < pos) { return; }
        // Se quiser fazer um update que soma, Altera aqui
        if(l == r) { Seg[no] = v; return; }
        int m = (l+r)/2, e = no*2, d = e+1;

        update(e, l, m, pos, v);
        update(d, m+1, r, pos, v);

        Seg[no] = join(Seg[e], Seg[d]);
    }

    void build(int no, int l, int r, vector<T> &Lista){
        if(l == r) {
            if(l < Lista.size()) { Seg[no] = Lista[l]; }
            else { Seg[no] = NEUTRO; }
            return;
        }
        int m = (l+r)/2, e = no*2, d = e+1;
        build(e, l, m, Lista);
        build(d, m+1, r, Lista);
        Seg[no] = join(Seg[e], Seg[d]);
    }

    T query(int ls, int rs) { return query(1, 0, N-1, ls, rs); }
    void update(int pos, T v) { update(1, 0, N-1, pos, v); }
};

int main() {
    vector<int> a = {5, 8, 2, 9, 3, 7};

    SegTree<int> Seg(a);

    // Query de 0 até 2
    Seg.query(0, 2);

    // Atualiza o idx 2 para 10
    Seg.update(2, 10);
}
```

04-FenwickTree.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

struct BIT {
    vector<ll> bit;
    int N;

    BIT() {}
    BIT(int n) : N(n+1), bit(n+1) {}

    void update(int pos, ll val) {
        for(; pos < N; pos += pos & (-pos)) { bit[pos] += val; }
    }
```

```
}  
// Quantos são menores ou iguais a pos  
ll query(int pos) {  
    ll sum = 0;  
    for(; pos > 0; pos -= pos & (-pos)) { sum += bit[pos]; }  
    return sum;  
}  
};
```

09-STRINGS

01-TRIE_.cpp

```
#include <bits/stdc++.h>
using namespace std;

// Memória: O(N * log(ALPHA) + M)
// Tempo: O((N + M) * log(ALPHA))

// Podemos lidar com mais letras além de a..z
const int ALPHA = 26;

// Ex: maiúsculas e minúsculas
int idx_trie(char c) {
    if('a' <= c && c <= 'z') { return c-'a'; }
    else if('A' <= c && c <= 'Z') { return c-'A' + 26; }
}

// nxt = próximos pontos que vem desse atual
// pref_qtd = quantas strings tem aquele prefixo
// str_qtd = quantas strings terminam ali
struct Node { int nxt[ALPHA] {}; int pref_qtd = 0; int str_qtd = 0; };

struct TRIE {
    vector<Node> tr;

    TRIE() : tr(1) {}

    void add_palavra(string s) {
        int cur = 0;

        for(char c : s) {
            int pos = c - 'a';
            if(tr[cur].nxt[pos] == 0) {
                int len = tr.size();
                tr[cur].nxt[pos] = len;
                tr.emplace_back();
            }

            tr[cur].pref_qtd += 1;
            cur = tr[cur].nxt[pos];
        }
        tr[cur].pref_qtd += 1;
        tr[cur].str_qtd += 1;
    }

    int query_prefix(string s) {
        int cur = 0;

        for(char c : s) {
            int pos = c - 'a';
            if(tr[cur].nxt[pos] == 0) { return 0; }
            cur = tr[cur].nxt[pos];
        }
        return tr[cur].pref_qtd;
    }

    int query_palavras(string s) {
        int cur = 0;

        for(char c : s) {
            int pos = c - 'a';
            if(tr[cur].nxt[pos] == 0) { return 0; }
            cur = tr[cur].nxt[pos];
        }
        return tr[cur].str_qtd;
    }
};

int main() {
    ios::sync_with_stdio(false); cin.tie(0);

    TRIE trie;
    trie.add_palavra("mesa");
    trie.add_palavra("mesada");
    trie.add_palavra("casebre");
    trie.add_palavra("casa");
    trie.add_palavra("casaco");
}
```

01-TRIE_zSamuel.cpp

```
#include <bits/stdc++.h>
using namespace std;

const int ALPHA = 26, off = 'a';
struct Node {
    array<Node*, ALPHA> nxt;
    int terminal = 0;
    Node() { nxt.fill(NULL); }
```

```
};

struct Trie {
    Node* root;
    Trie() { root = new Node(); }

    void add(string &s) {
        Node* t = root;
        for(auto c : s) { c -= off;
            if(!t->nxt[c])
                t->nxt[c] = new Node();
            t = t->nxt[c];
        }
        t->terminal++;
    }

    int count(string &s) {
        Node* t = root;
        for(auto c : s) { c -= off;
            if(!t->nxt[c]) return 0;
            t = t->nxt[c];
        }
        return t->terminal;
    }
};

/*LATEX_DESC_BEGIN*****
**Trie - Arvore de Prefixos**
insert(P) - O(|P|)
count(P) - O(|P|)
sigma - Tamanho do alfabeto
off - primeiro simbolo do alfabeto (offset)
*****LATEX_DESC_END*/
```

02.1-Hash.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
#define pii pair<int,int>
#define FORi(n) for(int i=0;i<n;i++)
#define FORj(n) for(int j=0;j<n;j++)
#define all(a) (a).begin(),(a).end()
#define printn(v,n) for(int i=0;i<n;i++) cout << v[i] << (i == n-1 ? "\n" : " ")

mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());

void solve() {
    int n, q; cin>>n>>q;

    vector<int> A(n), B(n);
    FORi(n) { cin>>A[i]; }
    FORi(n) { cin>>B[i]; }

    vector<ll> Random(n);
    FORi(n) { Random[i] = rng(); }

    vector<ll> Pre_Hash_A(n+1, 0), Pre_Hash_B(n+1, 0);
    FORi(n) {
        Pre_Hash_A[i+1] = Pre_Hash_A[i] + Random[A[i]-1];
        Pre_Hash_B[i+1] = Pre_Hash_B[i] + Random[B[i]-1];
    }

    FORi(q) {
        int l,r,L,R; cin>>l>>r>>L>>R;

        ll soma_A = Pre_Hash_A[r] - Pre_Hash_A[l-1];
        ll soma_B = Pre_Hash_B[r] - Pre_Hash_B[l-1];

        if(soma_A == soma_B) { cout << "Yes\n"; }
        else { cout << "No\n"; }
    }
}

int main() {
    ios::sync_with_stdio(false); cin.tie(0);

    solve();
}
```

02.2-Hash_2x.cpp

```
#include <bits/stdc++.h>
#include <random>
using namespace std;
#define ll long long

const ll MOD1 = 1000015553, MOD2 = 1000028537;
```

```

mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
static ll BASE = uniform_int_distribution<int>(356, MOD1-1)(rng);
//WA? Muda o MOD e a base

struct Hasher {
    string s;
    ll n;
    vector<ll> hash1, power1, hash2, power2;

    Hasher() : n(0) {}

    Hasher(string &s) : n(s.size()), s(s), hash1(n), power1(n), hash2(n), power2(n) {
        power1[0] = power2[0] = 1;
        for(int i=1;i<n;i++) {
            power1[i] = (BASE * power1[i-1]) % MOD1;
            power2[i] = (BASE * power2[i-1]) % MOD2;
        }

        hash1[0] = hash2[0] = s[0];
        for(int i=1;i<n;i++) {
            hash1[i] = (hash1[i-1] * BASE + (ll) (s[i])) % MOD1;
            hash2[i] = (hash2[i-1] * BASE + (ll) (s[i])) % MOD2;
        }
    }

    ll hash(int l, int r) {
        if(!l) { return (hash1[r] << 30) ^ (hash2[r]); }

        ll ans1 = ((hash1[r] - (hash1[l-1] * power1[r-l+1]) % MOD1 + MOD1) % MOD1;
        ll ans2 = ((hash2[r] - (hash2[l-1] * power2[r-l+1]) % MOD2 + MOD2) % MOD2;

        return (ans1 << 30) ^ (ans2);
    }
};

```

03.1-Hash_String.cpp

```

#include <bits/stdc++.h>
using namespace std;
#define ll long long
const int MAXN = 1e6 + 5;

const ll MOD = 1e9 + 7; //WA? Muda o MOD e a base
const ll base = 153;
ll expb[MAXN];

void precalc() {
    expb[0] = 1;
    for(int i=1; i<MAXN; i++) {
        expb[i] = (expb[i-1]*base)%MOD;
    }
}

struct StringHash {
    vector<ll> hsh;

    StringHash(string &s) {
        hsh.assign(s.size()+1, 0);
        for(int i=0; i<s.size(); i++) {
            hsh[i+1] = (hsh[i] * base % MOD + s[i]) % MOD;
        }
    }

    ll gethash(int l, int r) {
        return (MOD + hsh[r+1] - hsh[l]*expb[r-l+1] % MOD) % MOD;
    }
};

/*****
String Hash
precalc()    -> O(N)
StringHash() -> O(|S|)
gethash()    -> O(1)

StringHash hash(s); -> Cria uma struct de StringHash para a string s
hash.gethash(l, r); -> Retorna o hash do intervalo L R da string
(0-Indexado)

IMPORTANTE! Chamar precalc() no início do código

const ll MOD = 131'807'699; -> Big Prime Number
const ll base = 127;        -> Random number larger than the
Alphabet
*****/

```

03.2-Hash_String_2x.cpp

```

#include <bits/stdc++.h>
using namespace std;
#define ll long long
const int MAXN = 1e6 + 5;

const ll MOD1 = 131807699;
const ll MOD2 = 1e9 + 9;
const ll base = 157;

ll expb1[MAXN], expb2[MAXN];

#warning "Call precalc() before use StringHash"
void precalc() {
    expb1[0] = expb2[0] = 1;

    for(int i=1; i<MAXN; i++) {
        expb1[i] = expb1[i-1]*base % MOD1;
        expb2[i] = expb2[i-1]*base % MOD2;
    }
}

struct StringHash {
    vector<pair<ll, ll>> hsh;
    string s; // comment S if you dont need it

    StringHash(string& s) : s(s) {
        hsh.assign(s.size()+1, {0,0});

        for (int i=0; i<s.size(); i++) {
            hsh[i+1].first = ( hsh[i].first *base % MOD1 + s[i] ) % MOD1;
            hsh[i+1].second = ( hsh[i].second*base % MOD2 + s[i] ) % MOD2;
        }
    }

    ll gethash(int a, int b) {
        ll h1 = (MOD1+ hsh[b+1].first - hsh[a].first *expb1[b-a+1] % MOD1) % MOD1;
        ll h2 = (MOD2+ hsh[b+1].second - hsh[a].second*expb2[b-a+1] % MOD2) % MOD2;
        return (h1<<32) | h2;
    }
};

// OPTIONAL
int firstDiff(StringHash& a, int la, int ra, StringHash& b, int lb, int rb) {
    int l=0, r=min(ra-la, rb-lb), diff=r+1;
    while(l <= r) {
        int m = (l+r)/2;
        if(a.gethash(la, la+m) == b.gethash(lb, lb+m)) l = m+1;
        else r = m-1, diff = m;
    }
    return diff;
}

int hshComp(StringHash& a, int la, int ra, StringHash& b, int lb, int rb) {
    int diff = firstDiff(a, la, ra, b, lb, rb);
    if(diff > ra-la && ra-la == rb-lb) return 0; //equal
    if(diff > ra-la || diff > rb-lb) return ra-la < rb-lb ? -2 : +2;
    //prefix of the other
    return a.s[la+diff] < b.s[lb+diff] ? -1 : +1;
}

/*LATEX_DESC_BEGIN*****
**String Hash** - Double Hash
precalc()    -> O(N)
StringHash() -> O(|S|)
gethash()    -> O(1)

StringHash hash(s); -> Cria o Hash da string s
hash.gethash(l, r); -> Hash [L,R] (0-Indexado)
*****LATEX_DESC_END*/

int main() {
    ios::sync_with_stdio(false); cin.tie(0);

    precalc();
}

```

04-KMP.cpp

```

#include <bits/stdc++.h>
using namespace std;

// KMP_SAMUEL
// O KMP busca padrões em strings de forma linear O(N + M) sem nunca
retroceder o ponteiro do texto principal.

```

```
// Ele pré-processa o padrão em um vetor (pi) que identifica o maior
// prefixo que também é sufixo de cada subtrecho.
// Em caso de erro, esse vetor diz para onde o padrão deve "pular",
// aproveitando as comparações já feitas e evitando repetições inúteis.

vector<int> Pi(string &t){
    vector<int> p(t.size(), 0);

    for(int i=1, j=0; i<t.size(); i++){
        while(j > 0 && t[j] != t[i]) j = p[j-1];
        if(t[j] == t[i]) j++;
        p[i] = j;
    }
    return p;
}

vector<int> kmp(string &s, string &t){
    vector<int> p = Pi(t), occ;

    for(int i=0, j=0; i<s.size(); i++){
        while( j > 0 && s[i] != t[j]) j = p[j-1];
        if(s[i]==t[j]) j++;
        if(j == t.size()) occ.push_back(i-j+1), j = p[j-1];
    }
    return occ;
}

/*BLOCK_DESC_BEGIN **Optional:** KMP Automato. j = state atual
[root=j=0] BLOCK_DESC_END*/
struct Automato {
    vector<int> p;
    string t;
    Automato(string &t) : t(t), p(Pi(t)){}
    int next(int j, char c){ //return nxt state
        if(final(j)) j = p[j-1];
        while(j && c != t[j]) j = p[j-1];
        return j + (c == t[j]);
    }
    bool final(int j){ return j == t.size(); }
};

/*****
**KMP** - Knuth-Morris-Pratt Pattern Searching
Complexity: O(|S|+|T|)
kmp(s, t) -> returns all occurrences of t in s
p = Pi(t) -> p[i] = biggest prefix that is a suffix of t[0,i]
*****/
```

```
        cout << n/k << "\n" << s.substr(0, k) << "\n";
    }
    else {
        cout << "1\n" << s << "\n";
    }
}

int main() {
    ios::sync_with_stdio(false);cin.tie(0);

    solve();
}
```

05-PiArray.cpp

```
#include <bits/stdc++.h>
using namespace std;
#define FORi(n) for(int i=0;i<n;i++)

/**
 * pi[i] armazena o tamanho do MAIOR prefixo próprio de s[0..i]
 * que também é um sufixo dessa mesma substring s[0..i].
 *
 * - Complexidade: O(N) em tempo e O(N) em memória.
 * - Aplicações principais:
 * 1. Busca de padrão em texto (KMP matching).
 * 2. Identificação do menor período cíclico:
 *    k = N - pi[N - 1] (periódico se N % k == 0).
 * 3. Contagem de prefixos distintos e compressão de strings.
 */

vector<int> Pi(string &s) {
    int n = s.size();
    vector<int> pi(n, 0);

    for(int i=1;i<n;i++) {
        int j = pi[i-1];
        while(j > 0 && s[i] != s[j]) { j = pi[j-1]; }
        if(s[i] == s[j]) { j++; }
        pi[i] = j;
    }
    return pi;
}

// ENCONTRA O MENOR PREFIXO QUE PODE SER REPETIDO
// PRA GERAR A STRING INTEIRA

void solve() {
    string s; cin>>s;
    int n = s.size();

    vector<int> PiArray = Pi(s);

    int k = n - PiArray[n-1];
    if(n%k == 0) {
        // n/k repetições da substring
```

10-GEOMETRIA

01-Pontos.cpp

```
#include <bits/stdc++.h>
using namespace std;
#define FORi(n) for(int i=0;i<n;i++)

// tratar os Points como ponto flutuante apenas se for necessario,
// senao faz com int ou ll

const double EPS = 1e-9;
int cmp(double a, double b) {
    if(abs(a-b) < EPS) { return 0; } // São "iguais", se dif < EPS
    return (a < b) ? -1 : 1; // Retorna -1 se a < b, e 1 se a > b
}

struct Point {
    double x,y;
    Point(double x = 0, double y = 0) : x(x),y(y){}
    Point(const Point& p) : x(p.x), y(p.y){}

    bool operator < (const Point &p) const {
        if(cmp(x, p.x) == -1) return true;
        else if(cmp(x, p.x) == 1) return false;
        else { return cmp(y, p.y) < 0; }
    }

    bool operator == (const Point &p) const {return !cmp(x, p.x) && !cmp(y, p.y);}
    bool operator != (const Point &p) const {return !(p == *this);}

    // basic ops
    Point operator + (const Point& p) const {return Point(x + p.x, y + p.y);}
    Point operator - (const Point& p) const {return Point(x - p.x, y - p.y);}
    Point operator * (const double k) const {return Point(x*k, y*k);}
    Point operator / (const double k) const {return Point(x/k, y/k);}
};

// Produto Escalar => Se > 0: ângulo agudo (<90°); Se == 0:
// perpendiculares; Se < 0: ângulo obtuso (>90°).
double dot(const Point& p, const Point& q) { return p.x*q.x + p.y*q.y; }

// Produto Vetorial (Cross Product): Retorna a área do paralelogramo
// formado pelos vetores.
// Se > 0: q está à esquerda de p; Se == 0: colineares; Se < 0: q
// está à direita de p (sentido horário).
double cross(const Point& p, const Point& q) { return p.x*q.y - p.y*q.x; }

// Norma (Magnitude/Comprimento): Retorna o tamanho do vetor a
// partir da origem (0,0).
double norm(const Point& p) { return sqrt(p.x*p.x + p.y*p.y); }

// Distância entre 2 pontos
double dist(const Point& p, const Point& q) { return norm(p-q); }

// Distância ao quadrado entre 2 pontos
double dist2(const Point& p, const Point& q) { return dot(p-q, p-q); }

// Normalizar vetor (Vetor Unitario): Mantém a direção do vetor, mas
// define seu comprimento como 1.
Point normalize(const Point &p) { return p / norm(p); }

// Angulo entre 2 vetores: Retorna o ângulo orientado (com sinal) de
// p até q, variando de -PI a PI.
double angle(const Point& p, const Point& q) { return atan2(cross(p, q), dot(p, q)); }

// Angulo com a origem: Retorna o ângulo absoluto do ponto em
// relação ao eixo X positivo (coordenada polar).
double angle(const Point& p) { return atan2(p.y, p.x); }

// Rotaciona um vetor no sentido horário por um ângulo 'ang' (em
// radianos)
Point rotate_horario(const Point &p, double ang) {
    Point q;
    q.x = p.x * cos(ang) + p.y * sin(ang);
    q.y = -p.x * sin(ang) + p.y * cos(ang);
    return q;
}
Point rotate_antihorario(const Point &p, double ang) { return rotate_horario(p, -ang); }
```

```
// Area com Sinal
double sig_area(Point a, Point b, Point c) {
    return cross(b-a, c-b);
}

// Area do Polígono de Vertices em V
double pol_area(vector<Point> V) {
    int n = V.size();
    double soma = 0;
    for(int i = 0; i < n; i++) {
        soma += sig_area(Point(0,0), V[i], V[(i+1) % n]);
    }
    return abs(soma) / 2.0;
}

int main() {
    ios::sync_with_stdio(false);cin.tie(0);

    Point p1(10.5, 20.0); // x = 10.5, y = 20.0
    Point p2(5); // x = 5.0, y = 0.0 (o y assume o
// default)
    Point p3; // x = 0.0, y = 0.0 (ambos default)
    Point p4 = {3.0, 4.0};

    // DOT PRODUCT (PRODUTO ESCALAR)
    // Mede alinhamento entre vetores:
    // a · b > 0 -> mesmo sentido
    // a · b = 0 -> perpendiculares
    // a · b < 0 -> sentidos opostos

    // CROSS PRODUCT (PRODUTO VETORIAL)
    // Checar posição
    // a x b > 0 -> anti-horário
    // a x b < 0 -> horário
    // Resulta no vetor perpendicular à ambos
    // Módulo do vetor é igual à área do paralelogramo

    // PERGUNTAS:
    // Como saber se dois vetores são colineares? - R: cross(u,v) == 0
    // ou cmp(cross, 0) == 0
    // Como calcular a área do triângulo formado entre dois vetores? -
    // R: cross(u,v)/2 -> Metade do Módulo do vetor
    // Dados 3 pontos, como saber se eles são colineares? - R: Cria um
    // vetor de A -> B e A -> C e vê se são colineares
```

02-Convex_Hull.cpp

```
#include <bits/stdc++.h>
using namespace std;
#define all(a) (a).begin(),(a).end()

// Dado um conjunto de pontos, ache o menor
// polígono convexo que abrange todos os pontos

const double EPS = 1e-9;
int cmp(double a, double b) {
    if(abs(a-b) < EPS) { return 0; } // São "iguais", se dif < EPS
    return (a < b) ? -1 : 1; // Retorna -1 se a < b, e 1 se a > b
}

struct Point {
    double x,y;
    Point(double x = 0, double y = 0) : x(x),y(y){}
    Point(const Point& p) : x(p.x), y(p.y){}

    bool operator < (const Point &p) const {
        if(cmp(x, p.x) == -1) return true;
        else if(cmp(x, p.x) == 1) return false;
        else { return cmp(y, p.y) < 0; }
    }

    bool operator == (const Point &p) const {return !cmp(x, p.x) && !cmp(y, p.y);}
    bool operator != (const Point &p) const {return !(p == *this);}

    // basic ops
    Point operator + (const Point& p) const {return Point(x + p.x, y + p.y);}
    Point operator - (const Point& p) const {return Point(x - p.x, y - p.y);}
    Point operator * (const double k) const {return Point(x*k, y*k);}
    Point operator / (const double k) const {return Point(x/k, y/k);}
};

double cross(const Point& p, const Point& q) { return p.x*q.y - p.y*q.x; }

// PELO MENOS 3 PONTOS DISTINTOS, SE PONTOS FOREM TODOS COLINEARES O
// CONVEX_HULL TERA AREA = 0
```

```

vector<Point> convexHull(vector<Point>& pts, bool sorted = false) {
    int n = pts.size();
    if(!sorted) { sort(all(pts)); }

    vector<Point> lower(n+1), upper(n+1);
    int s = 0;
    for(int i=0; i<n; i++){
        lower[s++] = pts[i];
        while(s >= 3) {
            Point a = lower[s-3], b = lower[s-2], c = lower[s-1];
            Point v1 = b-a, v2 = c-b;
            // coloca >= se quiser os colineares
            if(cross(v1,v2) > 0) { break; }
            lower[s-2] = lower[s-1];
            s--;
        }
    }
    lower.resize(s);
    s = 0;
    for(int i=0; i<n; i++) {
        upper[s++] = pts[i];
        while(s >= 3) {
            Point a = upper[s-3], b = upper[s-2], c = upper[s-1];
            Point v1 = b-a, v2 = c-b;
            // coloca <= se quiser os colineares
            if(cross(v1,v2) < 0) { break; }
            upper[s-2] = upper[s-1];
            s--;
        }
    }
    upper.resize(s-1);
    reverse(all(upper));
    upper.pop_back();
    lower.insert(end(lower), begin(upper), end(upper));
    return lower;
}

bool isInside(const vector<Point> &hull, Point pt) {
    int n = hull.size();
    Point v0 = pt - hull[0], v1 = hull[1] - hull[0], v2 = hull[n-1] - hull[0];
    if(cross(v0,v1) > 0 || cross(v0,v2) < 0) { return false; }

    int l = 1, r = n-1;
    while(l != r) {
        int mid = (l + r + 1) / 2;
        v0 = pt - hull[0]; v1 = hull[mid] - hull[0];

        if(cross(v0,v1) < 0) { l = mid; }
        else { r = mid-1; }
    }
    v0 = hull[(l+1)%n] - hull[l], v1 = pt - hull[l];
    // >= 0 considera a Borda, > 0 considera apenas Dentro
    return cross(v0,v1) >= 0;
}

int main() {
    int n;
    vector<Point> pts(n);
    vector<Point> Hull = convexHull(pts);

    if(isInside(Hull, pts[0])) { cout << "Ponto esta dentro!\n"; }
    else { cout << "Ponto esta fora!\n"; }
}

```

03-ponto_no_poligono.cpp

```

#include <bits/stdc++.h>
using namespace std;

struct Point {
    double x,y;
    Point(double x = 0, double y = 0) : x(x),y(y){}
    Point(const Point& p) : x(p.x), y(p.y){}
};

// O(n) - n vertices do poligono

// Retorna: 1 (Dentro), 0 (Fora), -1 (Na Borda)
int pointInPolygon(Point &p, vector<Point> &poly) {
    int n = poly.size();
    bool inside = false;

    for (int i = 0; i < n; i++) {
        Point a = poly[i];
        Point b = poly[(i + 1) % n];

        // 1. Checa se o ponto está exatamente sobre a aresta AB
        double cross = (b.x - a.x) * (p.y - a.y) - (b.y - a.y) * (p.x - a.x);
        if (abs(cross) < 1e-9) {

```

```

            if (min(a.x, b.x) <= p.x && p.x <= max(a.x, b.x) &&
                min(a.y, b.y) <= p.y && p.y <= max(a.y, b.y)) {
                return -1; // Na borda
            }
        }

        // 2. Ray casting horizontal para a direita
        bool condA = a.y <= p.y;
        bool condB = b.y <= p.y;

        // Aresta cruza a linha horizontal de p.y
        if (condA != condB) {
            // Calcula a coordenada X da interseção
            double x_int = a.x + (p.y - a.y) * (b.x - a.x) / (b.y - a.y);

            if (x_int > p.x) {
                inside = !inside;
            }
        }
    }

    return inside ? 1 : 0;
}

```

04-triangulacao_poligono.cpp

```

#include <bits/stdc++.h>
using namespace std;

const double EPS = 1e-9;

struct Point {
    double x, y;
    Point(double x = 0, double y = 0) : x(x),y(y){}
    Point(const Point& p) : x(p.x), y(p.y){}

    Point operator + (const Point& p) const {return Point(x + p.x, y + p.y);}
    Point operator - (const Point& p) const {return Point(x - p.x, y - p.y);}
    Point operator * (const double k) const {return Point(x*k, y*k);}
    Point operator / (const double k) const {return Point(x/k, y/k);}
};

struct Triangle {
    Point a, b, c;
};

double cross(const Point& p, const Point& q) { return p.x*q.y - p.y*q.x; }
double orient(Point a, Point b, Point c) { return cross(b-a, c-a); }

// Checa se o ponto P está estritamente dentro ou na borda do
// triângulo ABC (em CCW)
bool pointInTriangle(Point p, Point a, Point b, Point c) {
    double o1 = orient(a, b, p);
    double o2 = orient(b, c, p);
    double o3 = orient(c, a, p);
    return (o1 >= -EPS && o2 >= -EPS && o3 >= -EPS);
}

// Algoritmo de Ear Clipping - O(N^2)
// Recebe os vértices de um polígono simples e retorna a lista de
// triângulos
// PEGA UM POLIGONO QUE PODE SER NAO-CONVEXO E RETORNA TRIANGULOS
// QUE FORMAM ELE
vector<Triangle> triangulate(vector<Point> poly) {
    vector<Triangle> triangles;
    int n = poly.size();
    if (n < 3) return triangles;

    // 1. Garante ordem anti-horária (CCW)
    double signed_area = 0;
    for (int i = 0; i < n; i++) {
        signed_area += cross(poly[i], poly[(i + 1) % n]);
    }
    if (signed_area < 0) {
        reverse(poly.begin(), poly.end());
    }

    // 2. Cria lista duplamente ligada circular via índices
    vector<int> prev(n), next(n);
    for (int i = 0; i < n; i++) {
        prev[i] = (i - 1 + n) % n;
        next[i] = (i + 1) % n;
    }

    int curr = 0;
    int remaining = n;

```

```

int safety_counter = 0;

// 3. Corta as orelhas (ears) sucessivamente
while (remaining > 2 && safety_counter < 2 * remaining) {
    int p = prev[curr];
    int nxt = next[curr];

    Point a = poly[p];
    Point b = poly[curr];
    Point c = poly[nxt];

    // Vértice 'b' precisa ser convexo para formar uma orelha
    if (orient(a, b, c) > EPS) {
        bool is_ear = true;

        // Checa se nenhum outro vértice restante está dentro de
        ABC
        for (int i = next[nxt]; i != p; i = next[i]) {
            if (pointInTriangle(poly[i], a, b, c)) {
                is_ear = false;
                break;
            }
        }

        // Se for uma orelha válida, remove o vértice 'curr'
        if (is_ear) {
            triangles.push_back({a, b, c});
            next[p] = nxt;
            prev[nxt] = p;
            remaining--;
            safety_counter = 0;
            curr = nxt;
            continue;
        }
    }

    curr = next[curr];
    safety_counter++;
}

return triangles;
}

```

```

// Corta um polígono convexo mantendo os pontos à esquerda da reta A
-> B
vector<Point> clipPolygon(const vector<Point>& poly, Point a, Point
b) {
    vector<Point> result;
    int n = poly.size();
    if (n == 0) return result;

    for (int i = 0; i < n; i++) {
        Point p1 = poly[i];
        Point p2 = poly[(i + 1) % n];

        double d1 = orient(a, b, p1);
        double d2 = orient(a, b, p2);

        if (d1 >= -EPS) { // p1 está dentro (ou na borda)
            result.push_back(p1);
            if (d2 < -EPS) { // p2 está fora (saiu)
                result.push_back(lineIntersection(a, b, p1, p2));
            }
        } else if (d2 >= -EPS) { // p1 estava fora, p2 entrou
            result.push_back(lineIntersection(a, b, p1, p2));
        }
    }
    return result;
}

double triangleIntersectionArea(Triangle t1, Triangle t2) {
    // 1. Garante que t2 esteja em ordem anti-horária (CCW)
    if (orient(t2.a, t2.b, t2.c) < 0) {
        swap(t2.b, t2.c);
    }

    // 2. Inicia com os vértices do triângulo 1
    vector<Point> subject = {t1.a, t1.b, t1.c};

    // 3. Corta o triângulo 1 sucessivamente pelas 3 arestas de t2
    subject = clipPolygon(subject, t2.a, t2.b);
    subject = clipPolygon(subject, t2.b, t2.c);
    subject = clipPolygon(subject, t2.c, t2.a);

    // 4. Retorna a área do polígono resultante
    return pol_area(subject);
}

```

05-area_entre_triangulos.cpp

```

#include <bits/stdc++.h>
using namespace std;

const double EPS = 1e-9;

struct Point {
    double x, y;
    Point(double x = 0, double y = 0) : x(x), y(y){}
    Point(const Point& p) : x(p.x), y(p.y){}

    Point operator + (const Point& p) const {return Point(x + p.x, y
+ p.y);}
    Point operator - (const Point& p) const {return Point(x - p.x, y
- p.y);}
    Point operator * (const double k) const {return Point(x*k, y*k);
}
    Point operator / (const double k) const {return Point(x/k, y/k);
}
};

double cross(const Point& p, const Point& q) { return p.x*q.y - p.y*
q.x; }
double sig_area(Point a, Point b, Point c) { return cross(b-a, c-b);
}
double pol_area(vector<Point> V) {
    int n = V.size();
    double soma = 0;
    for(int i = 0; i < n; i++) {
        soma += sig_area(Point(0,0), V[i], V[(i+1) % n]);
    }
    return abs(soma) / 2.0;
}

double orient(Point a, Point b, Point c) { return cross(b-a, c-a); }

struct Triangle {
    Point a, b, c;
};

// Ponto de interseção entre a reta suporte AB e o segmento PQ
Point lineIntersection(Point a, Point b, Point p, Point q) {
    Point ab = b - a;
    Point pq = q - p;
    double t = cross(p - a, pq) / cross(ab, pq);
    return a + ab * t;
}

```

MISC

BinarySearch.cpp

```
int smallestSolution(int l, int r) {
    while(l < r) {
        int m = (l+r)/2;
        if(check(m)) r = m;
        else l = m + 1;
    }
    return l;
}

int biggestSolution(int l, int r) {
    while(r - l > 1) {
        int m = (l+r)/2;
        if(check(m)) l = m;
        else r = m;
    }
    return check(r) ? r : l;
}
```

Divisibilidade.txt

Divisibilidade por 2:

O último algarismo deve ser par (0, 2, 4, 6 ou 8).

Divisibilidade por 3:

A soma de todos os algarismos do número deve ser divisível por 3.

Divisibilidade por 4:

Os dois últimos algarismos devem ser 00 ou serem um número divisível por 4.

Divisibilidade por 5:

O último algarismo deve ser 0 ou 5.

Divisibilidade por 6:

Deve ser divisível por 2 e por 3 ao mesmo tempo.

Divisibilidade por 7 (Chika Ofili):

Isole o último algarismo do número e multiplique esse algarismo por 5,
Some esse resultado ao número restante (o número original sem o último algarismo).
Ex: 532
53 + 2.5 = 63
63: 6 + 3.5 = 21 -> recursivamente

Divisibilidade por 8:

Os três últimos algarismos devem ser 000 ou serem um número divisível por 8.

Divisibilidade por 9:

A soma de todos os algarismos do número deve ser divisível por 9.

Divisibilidade por 11:

A soma alternada dos algarismos (soma dos ímpares menos a soma dos pares, da direita para a esquerda) deve ser zero ou múltiplo de 11.
Ex: 1925

Divisibilidade por 13:

Multiplique o último algarismo por 4 e some ao restante do número. O resultado deve ser divisível por 13.

Divisibilidade por 17:

Multiplique o último algarismo por 5 e subtraia do restante do número. O resultado deve ser divisível por 17 (ou 0).

Divisibilidade por 19:

Multiplique o último algarismo por 2 e some ao restante do número. O resultado deve ser divisível por 19.

Divisibilidade por 20:

Deve terminar em 00, 20, 40, 60 ou 80.

Divisibilidade por 67:

$N = 10R + u$
 $20N = 20R + 20u$
 $20N = (201R - R) + 20u$
 $20N = 201R - (R - 20u)$
 $(R - 20u) = 3.67R - 20N$
O termo $20N$ será divisível por 67 apenas se N for divisível por 67 (já que 20 e 67 não compartilham fatores primos, pois 67 é primo).

Ex: 2345

$234 - 5 \cdot 20 = 134$

$13 - 4 \cdot 20 = -67$

se Res = Múltiplo de 67 (como 0, 67 ou -67) o número original é divisível por 67.

GF.txt

$(a+b)^n =$ Somatório de $i=0$ até n de $\text{binomial}(n, i) \cdot a^{n-i} \cdot b^i$
o elevado do a e do b podem ser trocados se necessário / $(n-i)$ por i

$(1-x)^{-n} =$ Somatório de $i=0$ até o infinito de $\text{binomial}(n+i-1, i) \cdot x^i$

JulianDay.cpp

```
#include <bits/stdc++.h>
using namespace std;

string dayOfWeek[] = {"Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"};
// converts Gregorian date to integer (Julian day number)
int dateToInt(int d, int m, int y) {
    return 1461 * (y + 4800 + (m - 14) / 12) / 4 + 367 * (m - 2 - (m - 14) / 12 * 12) / 12 - 3 * ((y + 4900 + (m - 14) / 12) / 100) / 4 + d - 32075;
}
// converts integer (Julian day number) to Gregorian date:
month/day/year
void intToDate(int jd, int &d, int &m, int &y) {
    int x, n, i, j;

    x = jd + 68569;
    n = 4 * x / 146097;
    x -= (146097 * n + 3) / 4;
    i = (4000 * (x + 1)) / 1461001;
    x -= 1461 * i / 4 - 31;
    j = 80 * x / 2447;
    d = x - 2447 * j / 80;
    x = j / 11;
    m = j + 2 - 12 * x;
    y = 100 * (n - 49) + i + x;
}
// converts integer (Julian day number) to day of week
string intToDay(int jd) { return dayOfWeek[jd % 7]; }
```

LatLong.cpp

```
#include <bits/stdc++.h>
using namespace std;

/*
Converts from rectangular coordinates to latitude/longitude and vice versa. Uses degrees (not radians).
*/
struct latlong { double r, lat, lon; };

struct rect { double x, y, z; };

latlong convert(rect& P) {
    latlong Q;
    Q.r = sqrt(P.x*P.x+P.y*P.y+P.z*P.z);
    Q.lat = 180/M_PI*asin(P.z/Q.r);
    Q.lon = 180/M_PI*acos(P.x/sqrt(P.x*P.x+P.y*P.y));

    return Q;
}

rect convert(latlong& Q) {
    rect P;
    P.x = Q.r*cos(Q.lon*M_PI/180)*cos(Q.lat*M_PI/180);
    P.y = Q.r*sin(Q.lon*M_PI/180)*cos(Q.lat*M_PI/180);
    P.z = Q.r*sin(Q.lat*M_PI/180);

    return P;
}
```

Random.cpp

```
// mt19937_64 se LL
mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
// Random_Shuffle
shuffle(v.begin(), v.end(), rng);
// Random number in interval
int randint = uniform_int_distribution(0, i)(rng);
double randomDouble = uniform_real_distribution(0, 1)(rng);
// bernoulli_distribution, binomial_distribution,
geometric_distribution
// normal_distribution, poisson_distribution,
exponential_distribution
```

TernarySearch.cpp

```
// R (Busca Geral)
for(int i = 0; i < LOG; i++){
    long double m1 = (A * 2 + B) / 3.0;
    long double m2 = (A + 2 * B) / 3.0;

    if(f(m1) > f(m2))
        A = m1;
    else
        B = m2;
}
ans = f(A);

// Z (Busca em Array)
while(B - A > 4){
    int m1 = (A + B) / 2;
    int m2 = (A + B) / 2 + 1;
    if(f(m1) > f(m2))
        A = m1;
    else
        B = m2;
}
ans = inf;
for(int i = A; i <= B; i++) ans = min(ans , f(i));
```

permutation-int.cpp

```
// converte uma permutação pra um idx unico
// O(n) mas o resultado pode ser gerado no range [0, n!-1]
// para n > 12 resultado precisa de LL

int permutation_to_int(vector<int> &p) {
    int mask = 0;
    int i = 0, r = 0;
    for(int x : p) {
        r = r * ++i + __builtin_popcountll(mask & ~(1LL << x));
        mask |= 1LL << x;
    }
    return r;
}

const int MAXN = 1e3 + 7, MOD = 1e9 + 7;
ll fact[MAXN];

void precompute() {
    fact[0] = 1;
    for(int i=1; i<MAXN; i++) { fact[i] = (fact[i-1] * i) % MOD; }
}

// Indice da permutacao 0-idx de [0, n!-1]
ll permutation_to_idx(vector<int> &p) {
    int n = p.size();
    if(n == 0) return 0;

    int min_val = *min_element(p.begin(), p.end());
    ll mask = (1LL << n) - 1;
    ll rank = 0;

    for(int i = 0; i < n; i++) {
        int x = p[i] - min_val;

        ll small_disp = mask & ((1LL << x) - 1);
        int cnt = __builtin_popcountll(small_disp);

        rank += cnt * fact[n - 1 - i];

        mask &= ~(1LL << x);
    }

    return rank;
}
```

STRESS_TEST

brute.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// BRUTE = SOLVE LENTA, MAS GARANTIA DE QUE ESTÁ CORRETA

void solve() {

}

int main(){
    ios::sync_with_stdio(false);cin.tie(0);

    solve();
}
```

gen.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
#define printn(v,n) for(int i=0;i<n;i++) cout << v[i] << (i == n-1 ? "\n" : " ")

// GERA A ENTRADA DA QUESTÃO
// PRINTA O QUE VAI SER USADO DE INPUT

mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());

void solve(int n){
    vector<int> V(n);
    for(int i=0;i<n;i++) { V[i] = uniform_int_distribution<int>(1,64)(rng); }

    // RANDOMIZAR O VECTOR
    // shuffle(V.begin(), V.end(), rng);

    cout << n << " 1\n";
    printn(V, n);

    ll a = uniform_int_distribution<ll>(1,n)(rng);
    ll b = uniform_int_distribution<ll>(1,n)(rng);

    if(a>b) { swap(a,b); }
    cout << a << " " << b << "\n";
}

int main(){
    ios::sync_with_stdio(false);cin.tie(0);

    int t = uniform_int_distribution<int>(1,64)(rng);
    solve(t);
}
```

solve.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// SOLVE = SOLVE A SER TESTADA

void solve() {

}

int main(){
    ios::sync_with_stdio(false);cin.tie(0);

    solve();
}
```

stressTest.sh

```
P = solve #mude pro filename do codigo
Q = brute #mude pro filename do brute [correto]
g++ ${P}.cpp -o sol -O2 || exit 1
g++ ${Q}.cpp -o brt -O2 || exit 1
g++ gen.cpp -o gen -O2 || exit 1
for ((i = 1; ; i++)) do
    echo $i
    ./gen $i > in
    ./sol < in > out
    ./brt < in > out2
```

```
if (! cmp -s out out2) then
    echo "--> entrada:"
    cat in
    echo "--> saida code:"
    cat out
    echo "--> saida brute:"
    cat out2
    break;
fi
done
```

```
# COMO RODAR -> ABRE O TERMINAL GIT E USA O COMANDO:
# bash stressTest.sh
```